

第十九届全国大学生智能汽车竞赛

技 术 报 告



学 校：西华大学

队伍名称：技术涡流

参赛队员：庞 城

吴丙楠

张 栋

带队教师：裴西宏，陈飞

关于技术报告和研究论文使用授权的说明

本人完全了解全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：庞城 吴丙楠 张栋

带队教师签名：裴西宏 陈飞

日 期：2024-8-10

目录

关于技术报告和研究论文使用授权的说明.....	II
引言	4
第一章 系统方案设计	5
1.1 比赛要求	5
1.2 车模和微控制器选择	5
1.3 传感器的选用	5
1.4 舵机选用	6
1.5 无刷电机选用	6
第二章 机械结构	6
2.1 前轮定位	7
2.1.1 前轮前束.....	7
2.1.2 主销内倾.....	8
2.2 悬架调节	9
2.3 防撞和辅助翅膀设计	9
2.4 车模整体结构以及电路位置安放.....	10
第三章 硬件电路设计	12
3.1 主控板.....	12
3.2 电源模块设计	12
3.3 驱动模块设计	13
3.3.1 MOS 预驱和 MOS 选取	14
3.4 按键板.....	15
3.5 遥控器设计	16
3.6 硅麦阵列模块设计	16
第四章 算法部分	17
4.1 整体控制框架.....	18
4.2GPS 方面	18
4.2.1 GPS 数据的处理.....	18
4.2.2 转向控制.....	19
4.2.3 速度决策及速度控制.....	20
4.3 声音处理	22
第五章 开发工具说明	23
车模技术检查表.....	23
参考文献	34
附录：核心代码.....	34

引言

全国智能汽车竞赛是以智能汽车为研究对象的创意性科技竞赛，是面向全国大学生的一种具有探索性工程实践活动，是教育部倡导的大学生科技竞赛之一。该竞赛以“立足培养，重在参与，鼓励探索，追求卓越”为指导思想。以主委会所指定的微控制器为核心模块，制作模型汽车按照规定道路行驶，以完成时间最短者为优胜。该竞赛融科学性、趣味性和观赏性为一体，是以迅猛发展、前景广阔的汽车电子为背景，涵盖自动控制、模式识别、传感技术、电子、电气、计算机、机械与汽车等多学科专业的创意性比赛。该竞赛规则透明，评价标准客观，坚持公开、公平、公正的原则，力求向健康、普及、持续的方向发展。

十九届全国智能汽车竞赛越野信标组别是该竞赛运用 GPS 模块和硅麦传感器的第三年。相对于前两年的比赛要求，今年不仅要求越野车的极速也要求声音信标的准确和稳定，对于小车的速度控制和循迹精度有了更高的要求。

本文介绍了技术涡流在十九届全国智能汽车竞赛极速越野组的设计方案。分别对于机械结构、电路、算法等介绍。阐述了我们的想法和思路，希望对各位车友提供帮助。

第一章 系统方案设计

1.1 比赛要求

越野信标组选择标准操场内的 50×100 米左右区域进行比赛。场地内随机布置信标灯。车模从出发区出发后，按照 lora 模块接收到的信号依次灭灯。

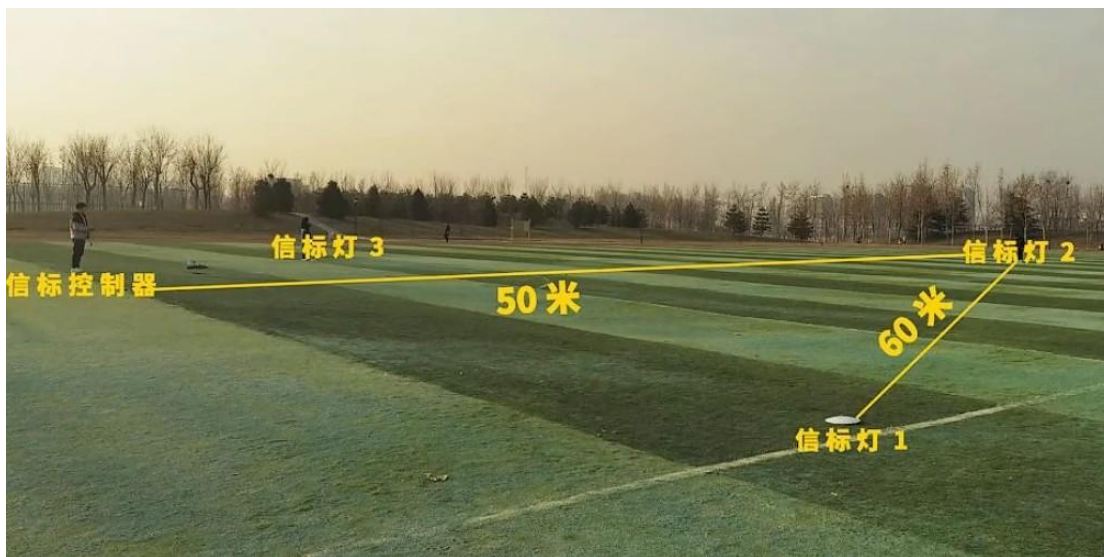


图 1 越野信标组比赛环境

1.2 车模和微控制器选择

本次设计采用的是全国大学生智能汽车竞赛越野信标组指定车模之一——R 车模，采用英飞凌公司的 CYT4BB7 芯片作为车模的主要可编程控制器。

1.3 传感器的选用

本次比赛允许使用多种传感器包括北斗-GPS、摄像头、惯性导航等传感器，我们采用 GPS 模块作为主要的传感器，陀螺仪模块辅助。

其中，GPS 采用逐飞出品的 GPS 模块方案，定位精度最高可达一米内，支持最高 10HZ 更新频率及 115200 波特率传输，实测能较好的满足比赛需求。陀螺仪采用 IMU660RA，计算小车的航向角和角速度，来实现转向，倒车以及直道行驶。

1.4 舵机选用

采用 BDS200 舵机来控制小车转向。其金属齿轮相比与塑料齿轮有更好的强度。

1.5 无刷电机选用

采用博思威龙 R 车 BDS3650 有感无刷电机，额定功率为 190w，电机转速为 3450kv（RPM/Volt）。

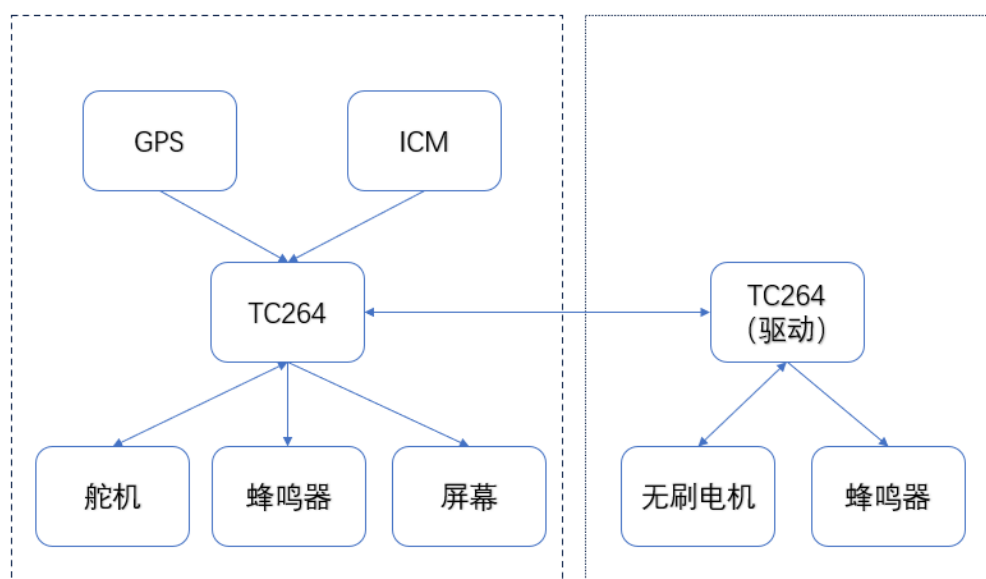


图 2 系统框架

第二章 机械结构

对于越野组而言，良好的车模机械结构决定其行进路线的好坏，如果没有一个巧妙并且稳定的结构，再优秀的算法也不能施展它应有的作用。各个模块都基于车模的结构基础之上，在实际情况下不断改进优化

和提高结构的稳定性，本章将主要介绍 R 车模的机械结构和调整方案。

2.1 前轮定位

2.1.1 前轮前束

前轮前束是指前轮前端面与后端面在汽车横向方向的距离差,也可指车身上前进方向与前轮平面之间的夹角,此时也称前束角。

当汽车两前轮的中心面不平行,两轮前边缘距离小于后边缘距离叫**负前束**;两轮前边缘距离大于后边缘距离叫**正前束**;相对应地相等就为 0 前束

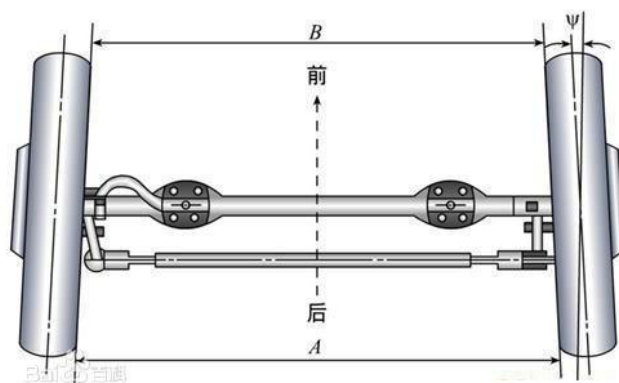


图 3 前轮前束

为保证汽车稳定的直线行驶,应使转向轮具有自动回正作用,即当转向轮在偶然遇到外力 (如碰到石块)作用发生偏转时,在外力消失后能立即自动回到直线行驶的位置。这种回正作用是由转向轮的定位参数来保证实现的。

由于前轮外倾,会使左右轮经常作用着方向相反的与外倾角相对应的侧倾推力,使轮胎磨损增加。为了避免这种情况,就要想办法消除这个侧倾推力。消除的办法,是在安装车轮时,使汽车两前轮的中心面不平行,两轮前边缘距离小于后边缘距离,两者的距离之差即为前轮前束。这样汽车在直行时左右轮就有一个大小相等方向和反的初始侧偏角。当然,这个初始侧偏角也会在左右轮胎印迹上造成方向相反的相应侧向力。

适当选择前束角,可使前束引起的侧向力与车轮外倾引起的侧倾推力相互抵消,从而避免了额外的轮胎磨耗和动力的消耗。我们的方案采用正前束。



图 4 前轮示意图

2.1.2 主销内倾

从车前后方向看轮胎时，主销轴向车身内侧倾斜，该角度称为主销内倾角。当车轮以主销为中心回转时，车轮的最低点将陷入路面以下，但实际上车轮下边缘不可能陷入路面以下，而是将转向车轮连同整个汽车前部向上抬起一个相应的高度，这样汽车本身的重力有使转向车轮回复到原来中间位置的效应，因而方向盘复位容易。

此外，主销内倾角还使得主销轴线与路面交点到车轮中心平面与地面交线的距离减小，从而减小转向时驾驶员加在方向盘上的力，使转向操纵轻便，同时也可减少从转向轮传到方向盘上的冲击力。但主销内倾角也不宜过大，否则加速了轮胎的磨损。



图 4 R 车模主销内倾调节示意图

2.2 悬架调节

R 车模的最大的特点就是它特有的 4 个悬挂装置，悬挂装置可以减轻车模在前进过程中的颠簸，但是原版的悬挂装置又有它的问题，如果悬挂过于柔软（即悬挂的弹性系数较低），会影响车体的速度上限，但悬架过于硬（即悬挂的弹性系数较高）则会影响车体制动减速以及坡道落地时导致舵机转向不稳定。因此我们舍弃原装弹簧，更换了 304 不锈钢材质的弹簧悬挂。



图 5 悬架示意图

2.3 防撞和辅助翅膀设计

由于越野组别在室外进行且速度较快，故我们加入了防撞装置，防止左右前轮和前差速器外壳损坏以及增加心理安慰。同时可以为翅膀的安装提供位置，辅助灭灯。

我们在车头安装两根一米长碳素杆，粘上锡纸和反光贴，并且尽可能加宽翅膀，增加灭灯的可能性。同时我们要考虑到车模接近信标灯时候由于急减速，会导致车头被压死，可能会把信标灯创飞的情况，在防撞支架安装时尤其要注意安装位置和角度。

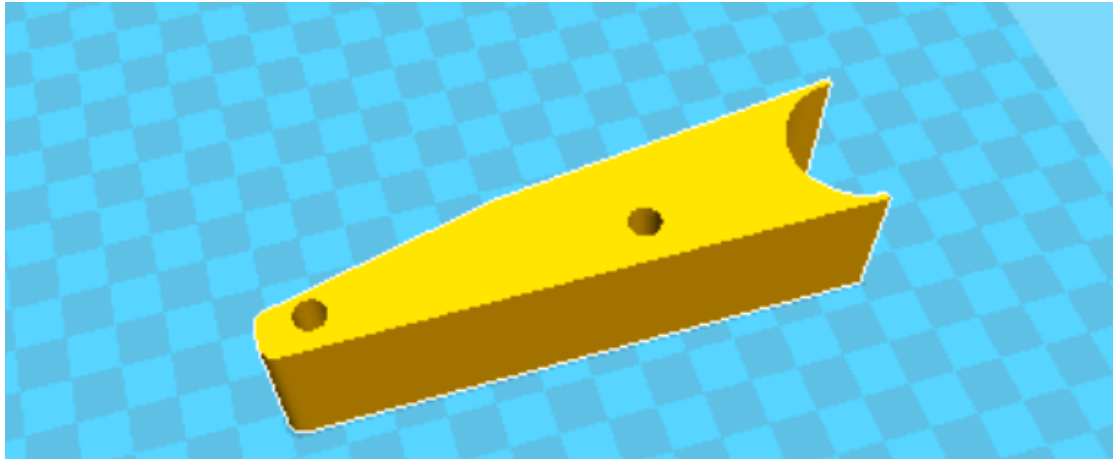


图 6 防撞杆示意图

2.4 车模整体结构以及电路位置安放

越野信标赛题使用的是 R 车模，R 车模需要自己改动的地方还有很多，为了更好的安装电路板和传感器，设计了一块能够安装各个模块的 3d 打印板，将与车模中的螺孔用铜柱连接。板子与板子之间用尼龙柱连接。电池和舵机、电机分别位于车模左右两侧以保证车模左右平衡。由于陀螺仪是用于感知车身整体姿态，所以我们将陀螺仪安装在了车身正中间，尽力放平并且固定牢固，尽量保证陀螺仪获取的数据准确可靠。GPS 天线也安装在了车头。由于我们小组的车模后期进行了改动，所以散热方面做得不是很好，没有安装上散热风扇，只是在电机和驱动上安装了散热片。由于越野组比赛期间车模的速度较高，车体上下流体压强的改变不能忽略，所以我们保留了 R 车模的尾翼装置。

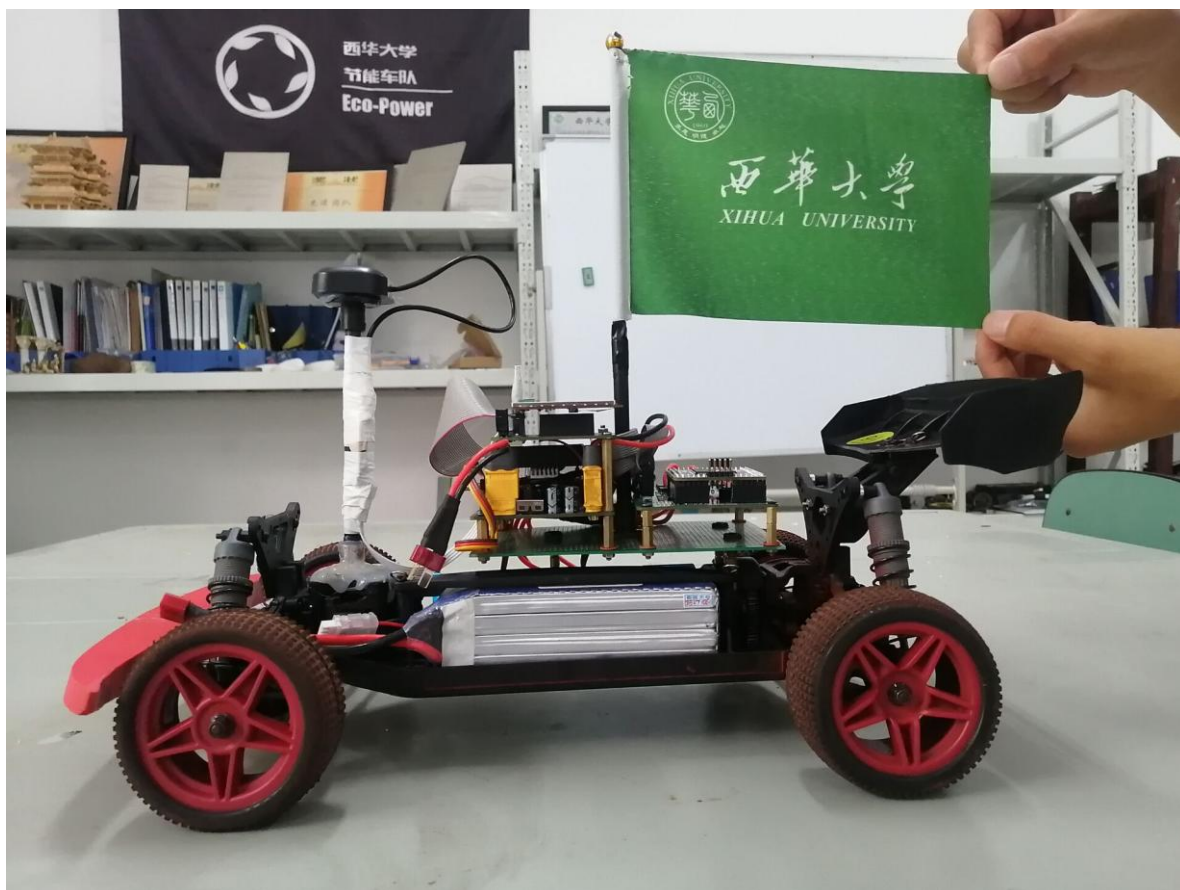


图 8 车模整体示意图

第三章 硬件电路设计

硬件电路是电路系统的重要组成部分，硬件电路设计是否合理直接影响电路系统的性能。硬件电路设计的一般分为设计需求分析、原理图设计、PCB 设计、工艺文件处理等几个阶段，设计过程中的每一个细节都可能成为导致设计成功与失败的关键。

我们智能车硬件电路设计由主控板、驱动板和按键板这三部分的设计组成

3.1 主控板

主控板以 CYT4BB 为核心，由电源模块以及系统各个模块接口组成（包括：舵机、陀螺仪、驱动板、硅麦板、GPS、串口等接口）。

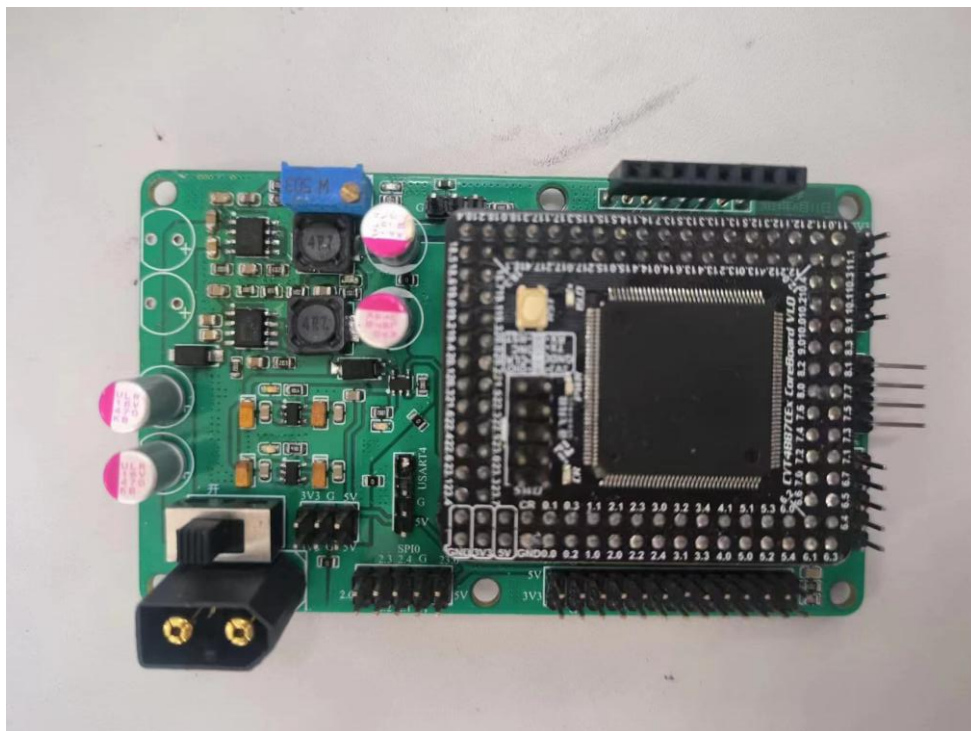
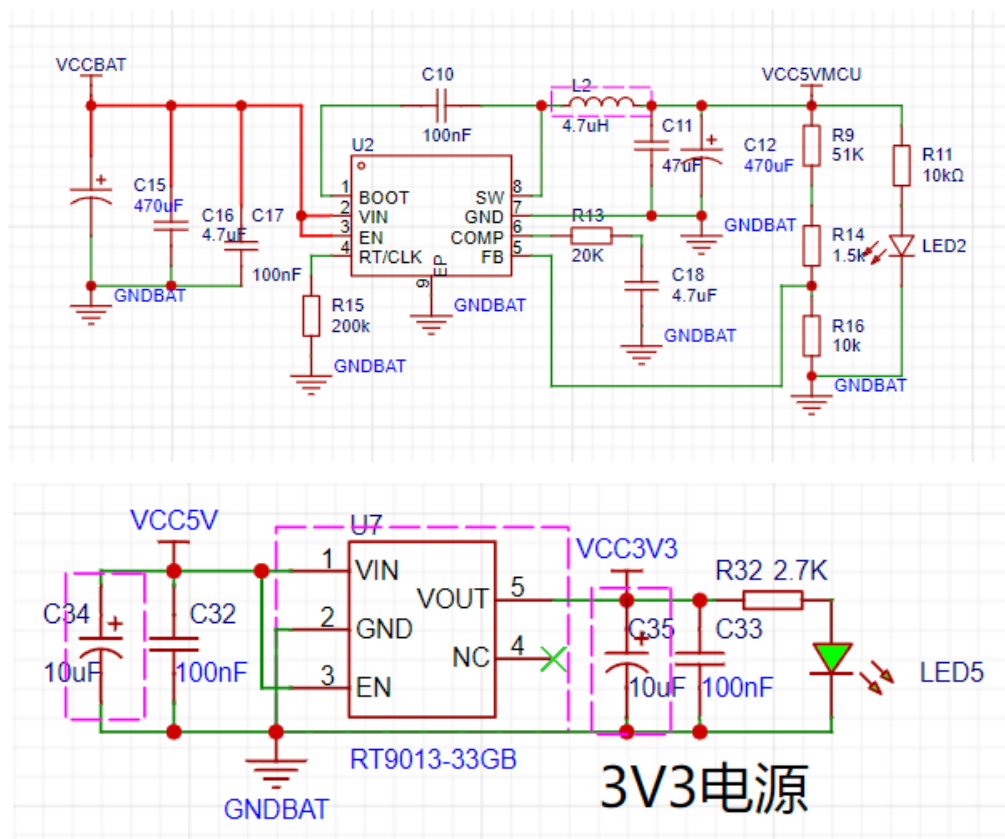


图 9 主控板

3.2 电源模块设计

电源模块对于一个控制系统来说极其重要，关系到整个系统是否能够正常工作，因此在设计控制系统时应选好合适的电源模块。

主板供电分有多个部分。我们采用 3s 电池，其最大电压约为 12.6V，经过 MC3406 稳定为 5V 后形成 5V 供电源，5V 再由 RT9013 稳定为 3.3V 形成 3.3V 供电源。其中舵机也单独安排了一个 MC3406 供电，舵机供电电压可通过电位器改变，这可以改变舵机的响应速度。核心板单独 5v 供电。所有供电模块都放置了一个 LED 灯作为电源指示灯，方便故障的排查。由于所有器件的供电都需要通过主板，为了方便电线的连接，我们合理的安排了所有接口的布局。



图

10 主板供电部分原理图

3.3 驱动模块设计

我们采用无刷电机进行驱动，且电机驱动的输出电流较高，故采用主板与驱动分离的形式，由于采用三相有感无刷电机，则需于驱动板上设计霍尔接口以及三相变换的 H 全桥。

无刷电机驱动设计的重点在于：栅极驱动器（MOS 预驱）和 MOS 管的选取以及栅极回路的 PCB 布线布局。这些皆为保证能够正确快速的开关 MOS 管，或者换句话说，得到理想的栅极电压波形（快速的上升、下降沿，无过冲、振荡）。

3.3.1 MOS 预驱和 MOS 选取

正如前文所说，MOS 的开通关断需要迅速，以期减少 MOS 的开关损耗。本质上，开通关断 MOS 是对 Ciss 进行充放电。故尽量选取推拉电流高的栅极驱动器对栅极充放电。这里选用 IR2181 的国产平替 EG2181，输出电流能力 IO+/-2/2.5A 满足我们的需求。且自带死区保护，可避免出现上下桥直通的现象。

MOS 管尽量选用耐压高的型号。Rdson 较低可减少导通时的发热。Ciss 较小可以减轻振荡。不过由于制作工艺的限制，Rdson 和 Ciss 往往无法同时满足要求，故选取时应该权衡取舍。我们选用了 HYG006N04LS1TA 这款 MOS，Rdson 可低至 0.45 毫欧，Ciss 为 15400pF。

除此之外，栅极串联电阻的大小会影响开关时间，也可用于抑制栅极振荡。栅极电阻大，可有效减少振荡，同时也会增加栅极上升下降沿时长。其选值大小仍需权衡取舍。可更换电阻后用示波器观察栅极电压波形，比较后择优选取。

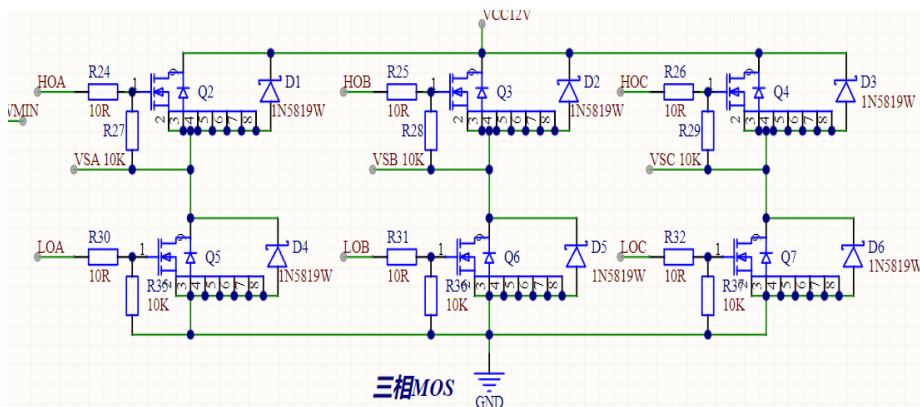
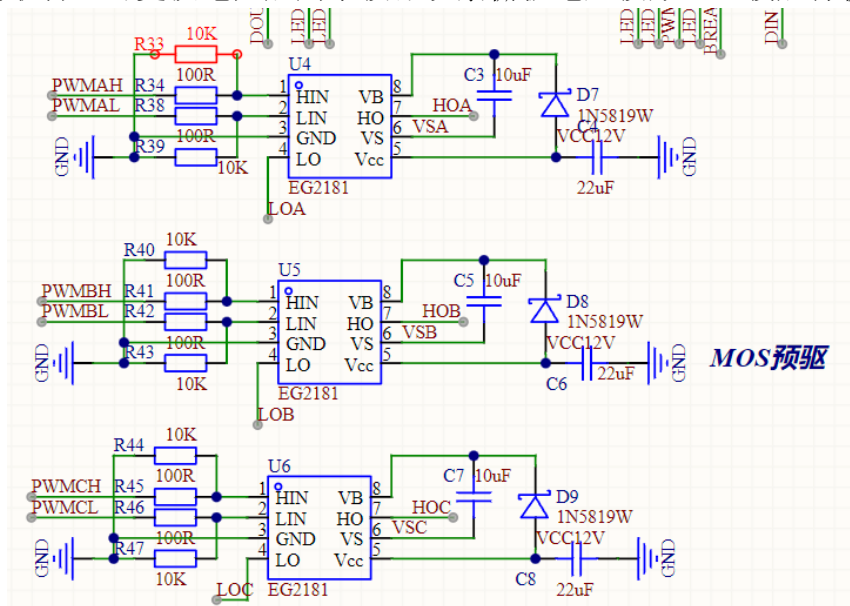


图 11 三相预驱电路

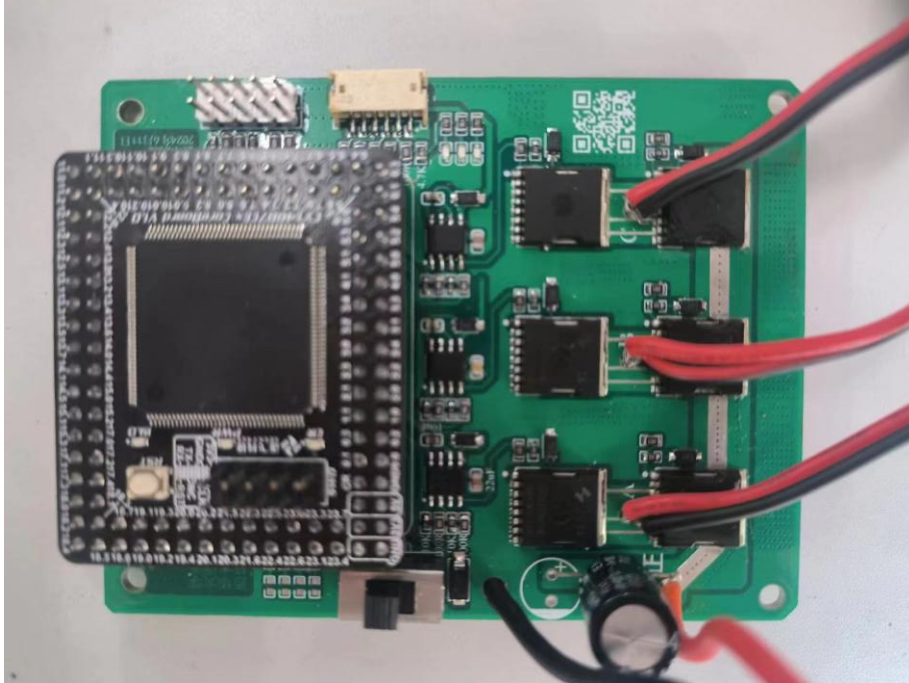


图 12 驱动板正面图

3.4 按键板

由于规则规定不能在比赛中下载程序，因此将按键和屏幕单独设计一块板子方便参数的调节。

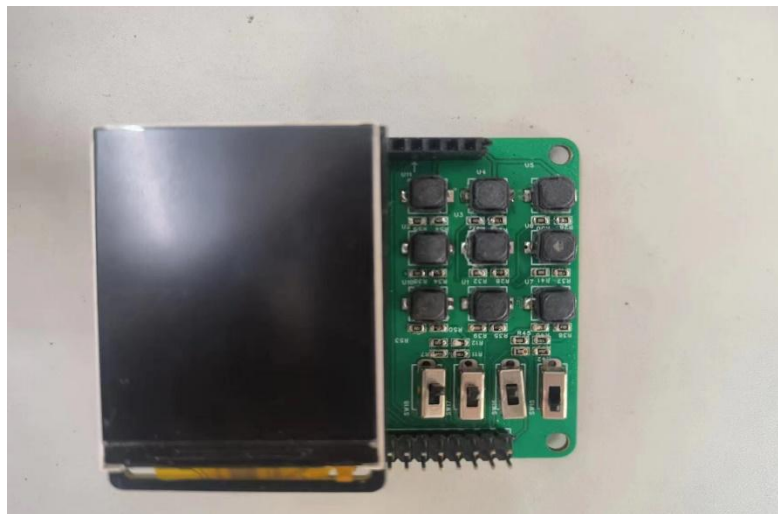


图 12 按键板

3.5 遥控器设计

我们利用常用德 stm32f103c8t6 芯片作为主控制作遥控器，在调试中如果发生意外通过无线串口发送数据给小车，使小车紧急刹车，防止事故的发生和车模的损坏。

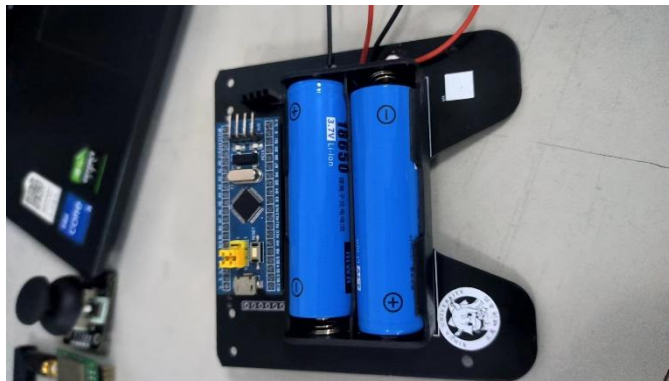


图 13 遥控器

3.6 硅麦阵列模块设计

本功能模块由三部分组成：提供接口和机械安装位置的安装板、板载信号放大电路的延长板，和板载 im68a130 的自制模块。在所有 PCB 都是白嫖尺寸下，通过这些 PCB 模块的组装，可使硅麦阵列长度最大化。且任意部位损坏都便于更换。

信号放大电路采用德州仪器的 LMV358 作为核心器件（LMV321 也可以），该器件支持 rail-to-rail 输出，可以在几乎 0~供电电压 3.3V 范围内任意输出，使得 ADC 采样范围得到最大的利用。

此外需注意 im68a130 自带约 1.3v 的直流偏置，应在计算好运放的静态工作点。为最大利用输出范围，避免超出输出范围造成的失真，将静态输出设置在 1.65V 左右。放大倍数可根据调试情况，更换 R7 和 R4 调整放大倍数。静态工作点通过 R7 和 R5 调整。

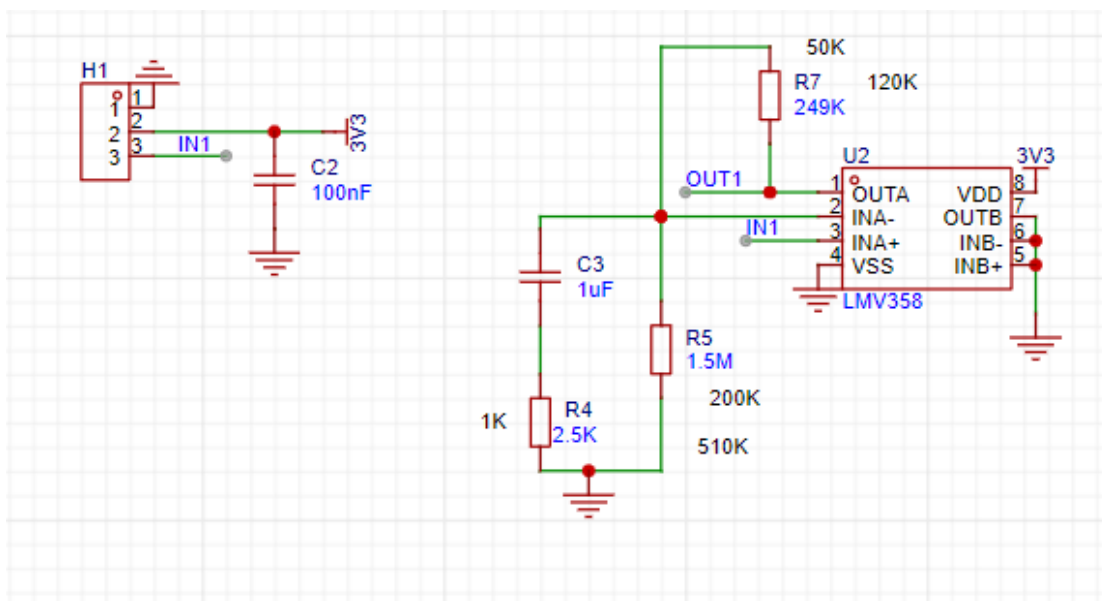


图 14.1 信号放大电路



图 14.2 组装完成的硅麦阵列

第四章 算法部分

控制程序是人的思想在车模体现，程序要体系化，模块化，稳定化，目标是将硬件电路和机械性能发挥到最大，让车模用最快的速度完成比赛。我们认为对于极速越野组，GPS 循迹和速度控制是这次比赛难点之处，故分为这两部分介绍。

4.1 整体控制框架

由于今年使用的芯片有三个核心，性能强劲。我们在其中一个 m7 的核心开启一个 100us 一次的定时器中断采集声音信号，并在主循环不断计算。同时在另一块 m7 核心进行 5ms 为周期的数据解算，包括 gps 数据解算，imu 的姿态解算，双核通讯接收声音解算结果和速度环。

同时我们在 5ms 的中断中通过记次数的方式实现 20ms 为周期的方向环和 100ms 为周期的速度决策。这样每个核心只占用一个定时器中断，但是要考虑到每 100ms 会有一次中断进行大量的计算，保证代码可以在 5ms 内跑完一轮。

4.2GPS 方面

今年越野车虽然和声音信标进行了融合，但是我们采用的北斗导航模块具有亚米级精度，所以我们认为 gps 的引导应该占据主导地位，在前期和后期我们都尝试过只采用 gps 导航的方式均可灭灯。

4.2.1 GPS 数据的处理

上面提到，北斗模块具有很高的精度，根据测试发现模块读取到的经纬度数据是非常准确的，所以我们并未对坐标进行处理。

但是模块返回的速度值和航向角需要在车模运行过程中才准确，我们认为其逻辑是模块根据本次和上次采集的两次坐标连成一条直线然后来计算航向角的，所以需要运动起来才可以得到准确的数值。同时模块的数据回传频率只有 10hz，结合模块的亚米级别精度，也就是速度大于 1m 的时候数据才具有可信度。这时候可以考虑加入陀螺仪来修正角度。

于是我们尝试引入地磁计来得到航向角，思路为：首先根据加速度和角速度进行姿态解算得到 pitch 和 roll 角度，为地磁极提供一个基准平面后融合得到 yaw，然后由于地磁计得到的是相对于地球磁极的角度，我们还要补偿到相对于正北坐标系的角度。我们可以查询所在的经纬度对应的地磁偏角或者用指南针手动测量地磁偏角进行补偿。最后实测仅用地磁计不适用模块返回的航向角也可正确循迹，但是容易受到赛道的颠簸影响和车模高速运行下的噪声影响，最后并没有采用地磁计方案。

我们也尝试过不使用地磁计仅使用陀螺仪再手动指北之后融合，但是上文提到，车模运行过程中的航向角已经很准确，这样融合也只能在启动瞬间保证一下，最后我们也没有采用这种方案。

4.2.2 转向控制

使用 GPS 循迹主要是利用车模当前的点位与存在程序里的目标点位进行对比，以正北为基准，算出目标航向角。同时解算出车模当前航向角，将目标航向角与当前航向角求差，即可得到目标偏差量。

```
void GPS_CAL()
{
    if(SW_12==0) //进入寻迹模式
    {
        azimuth=get_two_points_azimuth(gps_tau1201.latitude,gps_tau1201.longitude,temp[cur_posi].lat,temp[cur_posi].lon);

        err=azimuth-gps_tau1201.direction; //航向角接近方位角控制
        // err=azimuth-Yaw;

        if(err> 180 ) err= -360+err;
        if(err<-180 ) err= 360+err;
    }
}
```

图 15 偏差获取程序

将所测得偏差经过 PD 算法控制舵机实现 GPS 点位的循迹。

位置 PD，将偏差的比例（Proportion）和微分（Differential）通过线性组合构成控制量，用这一控制量对被控对象进行控制，这样的控制器称 PD 控制器。

$$y = p * er + d * (gyro)$$

式中 p 为比例项 d 为微分项

er 为偏差 $gyro$ 为 z 轴角速度

（1）微分项 d

一般的控制系统，不仅对稳定控制有要求，而且对动态指标也有要求，通常都要求负载变化或给定调整等引起扰动后，恢复到稳态的速度要快，因此光有比例和积分调节作用还不能完全满足要求，必须引入微分作用。比例作用和积分作用是事后调节，而微分作用则是事前预防控制，即一发现 y 有变大或变小的趋势，马上就输出一个阻止其变化的控制信号，以防止出现过冲或超调等。

d 越大，微分作用越强， d 越小，微分作用越弱。系统调试时通常把 d 从小往大调，具体参数由试验决定。

（2）比例项 p

p 控制按偏差 er 的比例关系对被控对象进行控制，偏差一旦产生，控制器立即产生控制作用，以减小偏差。当仅有比例控制时系统输出存在稳态误差。 p 参数越大，比例作用越强，动态响应越快，消除误差的能力越强。但实际系统是有惯性的，控制输出变化后，实际 y 值变化还需等待一段时间才会缓慢变化。由于实际系统是有惯性的，比例作用不宜太强，比例作用太强会引起系统振荡不稳定 p 参数的大小应在以上定量计算的基础上根据系统响应情况，进行调试决定，通

常将 P 参数由大向小调，以能达到最快响应又无超调为最佳参数。

```
52 duty=Kp*Value+Kd*gz;//陀螺仪z轴角速度正负遵循右手定则，逆时针+
53 duty=ServoMiddle+duty;
54
55 // if(cur_posi==0) //每跑一圈，标志位刷新
```

4.2.3 速度决策及速度控制

4.2.3.1 速度决策

由于车辆来说，速度大小会影响其转弯幅度的大小，速度越快其转弯幅度越大，对于小车直道入坡和调头有了很高考验。因此对于速度控制我们将舵机转角，目标点距离远近，小车速度这三个变量基于一次函数串到一起。公式如下：

$$\begin{cases} speed_{high} = speed_{low} + \frac{speed_{fast} - speed_{low}}{distance_{high} - distance_{low}} * (distance - distance_{low}) \\ speed_{want} = speed_{high} - \frac{speed_{high} - speed_{safe}}{duty_{max}} * |duty - duty_{middle}| \end{cases}$$

式中 $speed_{want}$ 为小车速度

$speed_{high}$ 为目标点距离远近所决定的最大速度

$speed_{low}$ 为目标点距离远近所决定的最低速度

$speed_{fast}$ 为小车最大速度

$speed_{safe}$ 为小车最低速度

$duty_{max}$ 为舵机最大偏差

$duty$ 为舵机值

$duty_{middle}$ 为舵机中值

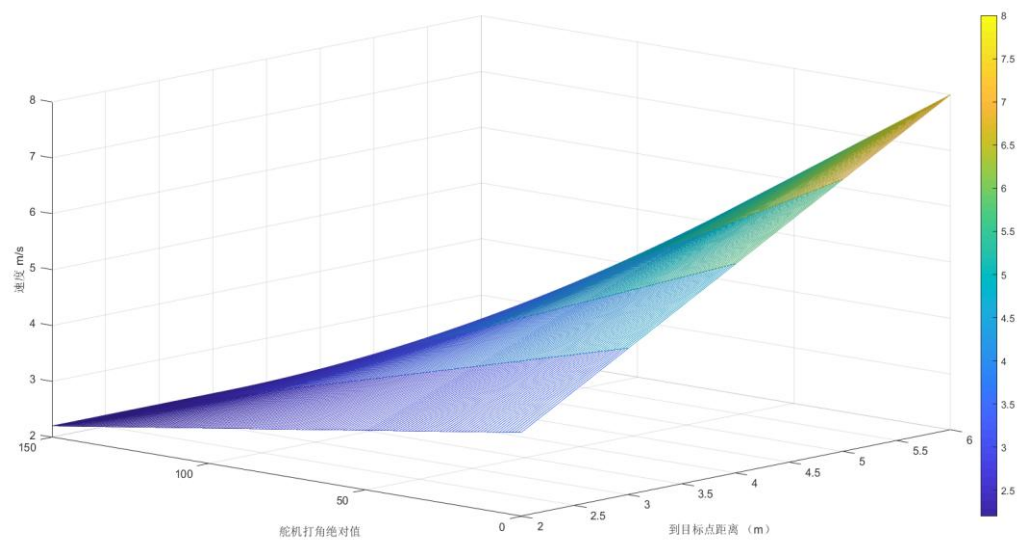


图 16 速度曲面

由速度曲面可以看出在速度最低处为小车调头处，而速度最高处为小车在直道行驶处。我们在调试发现该算法可以满足赛道上大部分速度控制。

4.2.3.2 速度控制

我们采用位置式 PID 对小车进行速度闭环控制，由于前面写了 PD 控制则只介绍 I 项。

I 控制器的输出与输入误差信号的积分成正比关系。I 控制主要用于消除静态误差。比例作用的输出与误差的大小成正比，误差越大，输出越大，误差越小，输出越小，误差为零，输出为零。由于没有误差时输出为零，因此比例调节不可能完全消除误差。必定会存在一个稳定的误差，以维持一个稳定的输出。这个控制误差称为静态误差，加强比例作用只能减少静态误差，但不能消除静态误差。为了消除静态误差，引入积分控制，积分作用可以消除静差，以使被控对象的状态 $y(t)$ 值最后与给定值一致。

```
if(SpeedInter> 150) SpeedInter= 150;  
if(SpeedInter<-60) SpeedInter=-60; //抗饱和  
  
SpeedOut=SpeedError*SpeedKp+index*SpeedInter*SpeedKi+SpeedDiffer*SpeedKd;  
  
SpeedOut=(SpeedOut> 9900? 9900:SpeedOut);  
SpeedOut=(SpeedOut<-9900?-9900:SpeedOut);
```

图 17 速度闭环控制

但是在后期我们发现了一个严重的问题，我们使用的无刷电机的驱动板会根据电机的转速返回一个频率变化的方波，然后主板就可以用定时器编码器模式捕获脉冲来测速。当电机转动过快后，驱动板的输出频率超过 16k 后将会导致测速环节测到错误的的数据，此时小车整个闭环都会错误，无法正常运行。

由于我们一开始使用的是十八届的 L 车模，L 车模的电机传动比较小，当车速到达八米每秒的时候，闭环就可能会出现错误，不能满足我们的需求，于是我们采用开环控制的策略：在做完速度决策之后，经目标速度乘以 900 当作输出占空比。但是我们又发现这样做很受 gps 和赛道的影响，如果 gps 不准或者赛道很滑很容易冲过信标灯无法有效减速。于是我才用棒棒算法，及在该减速的距离（我设定的是离灯 6m 时），直接输出 1700 占空比，让车轮保持滚动，与地面存在相对摩擦，可以稳定减速下来。

在后期我们更换成 R 车模，由于 R 车模电机齿轮传动比较大，在车速小于 15m/s 时都不出现闭环错误的情况，我们采用闭环操作。

4.3 声音处理

第五章 开发工具说明

AURIX™ Development Studio（以下简称 ADS）是英飞凌公司于 2019 年底推出的集成开发环境，支持英飞凌 TriCore™内核 AURIX™ 系列 MCU，ADS 是一个完整的开发环境，包含了 Eclipse IDE、C 编译器、Multi-core 调试器、英飞凌底层驱动库（low-level driver,iLLD),同时对于编辑、编译及调试应用代码没有时间及代码大小的限制。ADS 软件目前支持 Microsoft Windows10。ADS 除开发工具一条龙服务外，还配套有辅助性服务，如 Setup 集成化、在线屏幕手册 (Manual) 集成化、出错管理 (Error Handling) 集成化、FSS 窗口等。在此还需要感谢逐飞科技提供的 TC264 开源库，才使得备赛过程顺利进行。

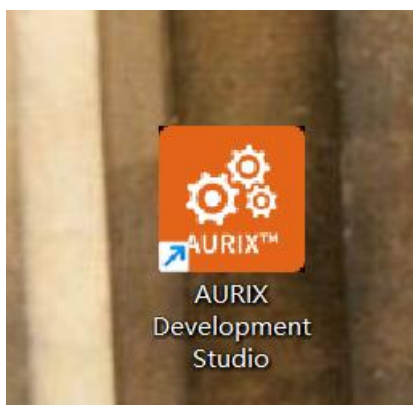


图 18 AURIX™ Development Studio

车模技术检查表

队伍名称	技术涡流			
参赛学校	西华大学			
赛题组组别	越野信标组			
检查项目	规格 (选手自行填写)	符合 (Ö)	不 符合 (')	备注

1. 车模类型是什么？	L 车模+R 车模	Ö		如果是自制车模，请标明自制。
车模整体尺寸： （包括传感器在内）长，宽，高(mm)	1000*450*350	Ö		今年比赛车模尺寸没有限制
传感器种类、规格(型号)数量。 是否使用自带 MCU 的成品传感器模块？型号是什么？	惯性导航单元 imu963ra 两个 硅麦模块 im68a130 八个 北斗导航模块两个	Ö		
控制转向舵机型号 是否自行改装舵机？ 防伪易损标签是否完整？	1 否 2 标签完整	Ö		
1. 是否增加伺服电机？ 2. 如果有种类、个数和作用？	否	Ö		
电路中微处理器型号和个数？	Cyt4bb7 四个	Ö		
是否具有其它可编程器件，个数与作用？	否	Ö		
是否有无线通讯装置？ 如果有种类和个数？	装有 LQ4C39 lora 模块两个	Ö		
电池的种类、规格和数量？	3s 锂电池 2600mah 2 个	Ö		
1. 是否使用 GPS 导航？ 2. 是否没有使用 RTK？	使用 gps 导航，未使用 RTK	Ö		
后轮驱动电机是否是原车模电机？ 是否具有防伪易损	1 是 2 是	Ö		

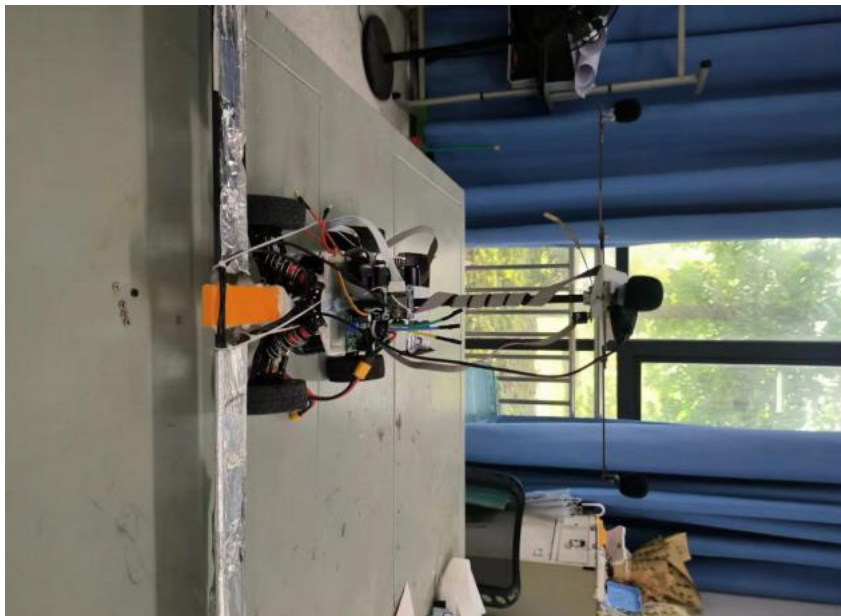
标签?				
车模轮胎是否原有的纹理可辨析? 轮胎表面是否具有粘性物质? 对于麦克纳姆轮是否更换过小轮胶皮?	1 纹理可辨析 2 否 3 否	Ö		
车轮轴距、轮距是否改装? 改装参数是什么?	否	Ö		
车模驱动轮传动机构是否改装? 改装方式是什么?	否	Ö		
是否更换过原装车模中的机械元器件? 更换后的规格 是什么?	更换了 R 车的减震弹簧 为 304 不锈钢, 线径 1.4 外径 16 长度 40	Ö		。
车模电路板个数及功能。 其中是否有购买成品电路板?	24 个 主板两个, 驱动板两个, 硅麦模块八个, 外接按键板两个, 硅麦支架两个, 硅麦放大电路八个 未使用成品电路板	Ö		
自制电路板是否标记有学校名称、队伍名称、制作日期等信息? 标示信息在 PCB 的哪一层?	有标记 顶层覆铜区	Ö		请在表格中注明电路板队伍信息的内容。
其它待说明内容				
检查人员签名:	检查意见:			

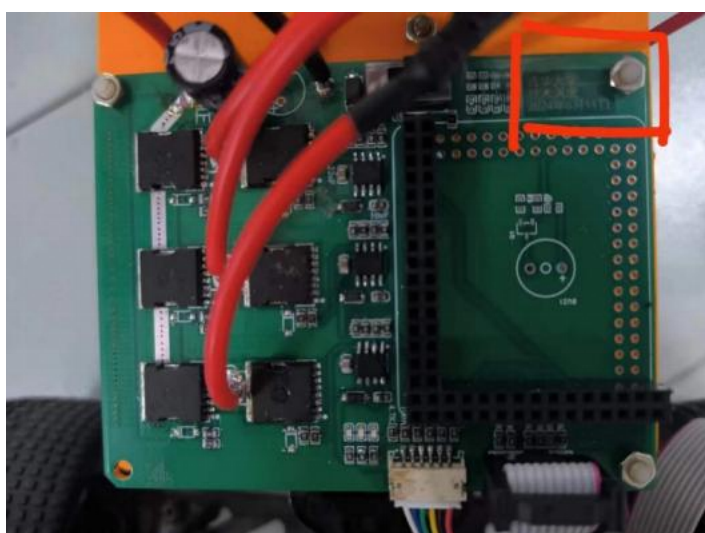
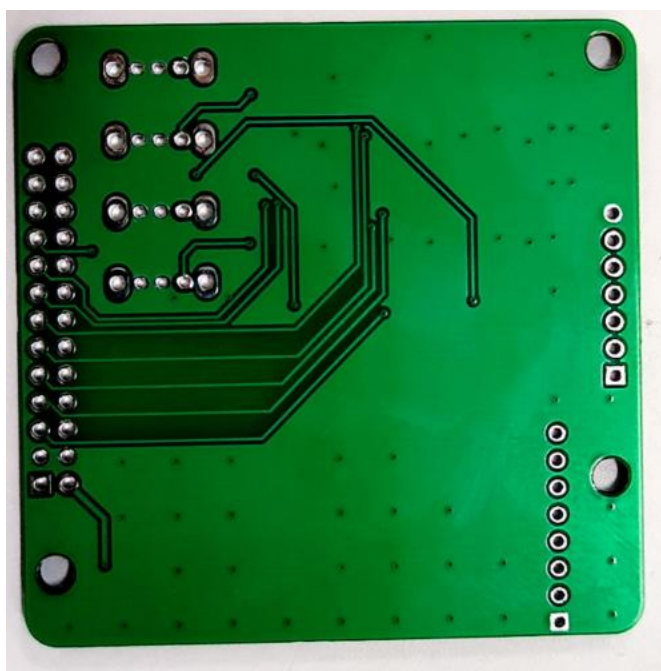
--	--

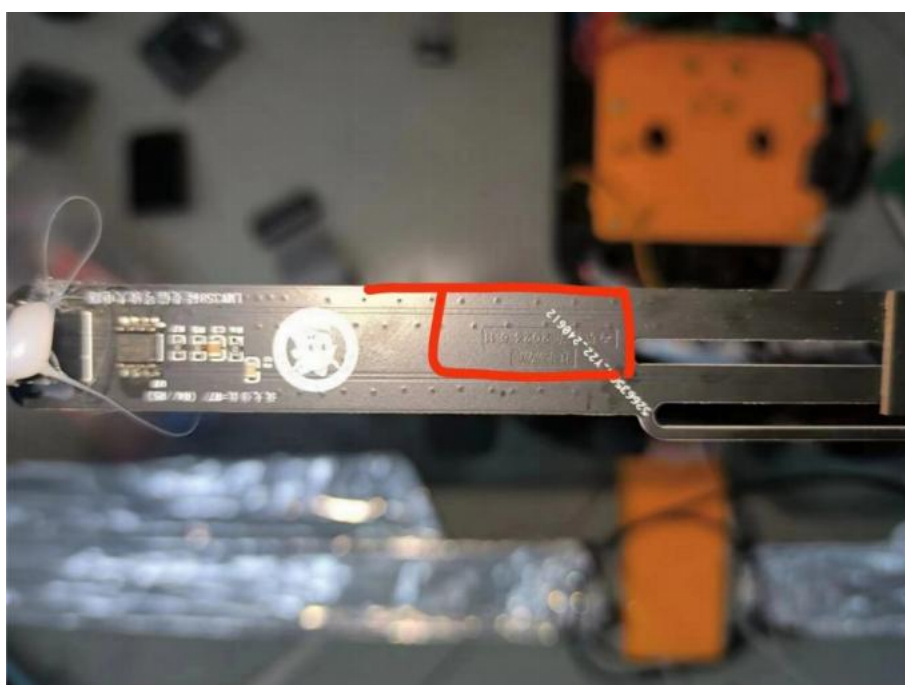
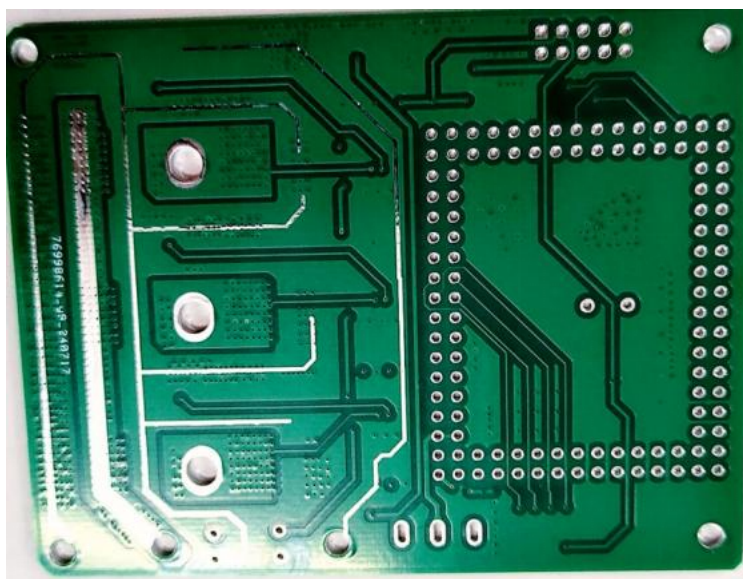
二、车模照片

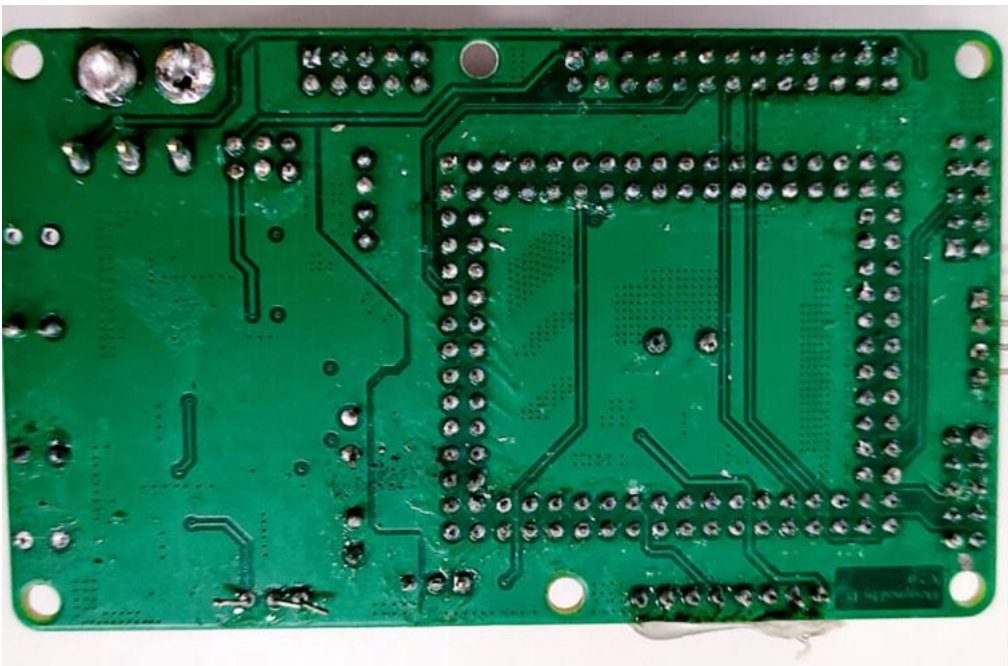
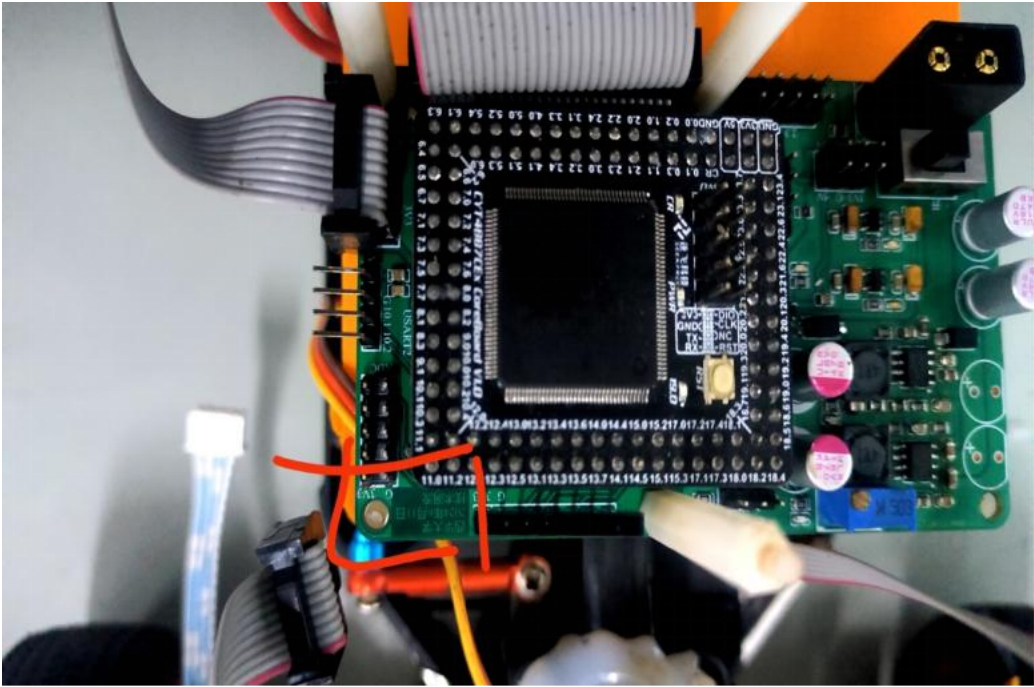
1、车模外观照片

车模平放时，俯视照片，前视图，左或右（任选）视图照片.









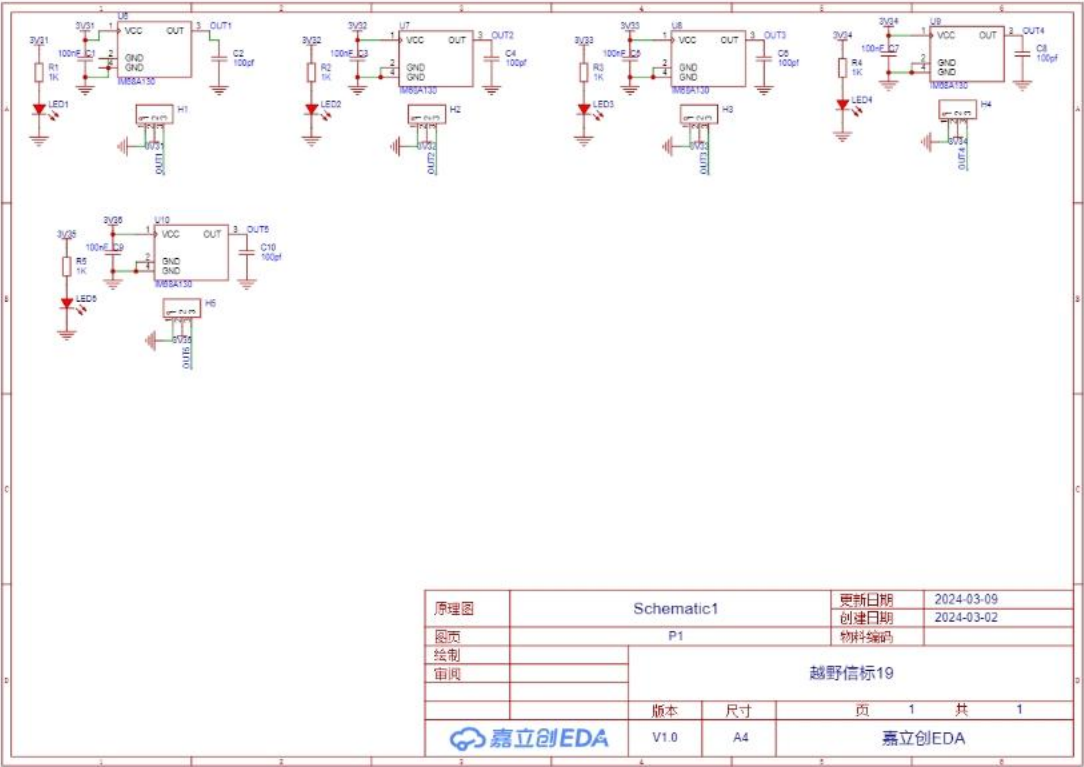
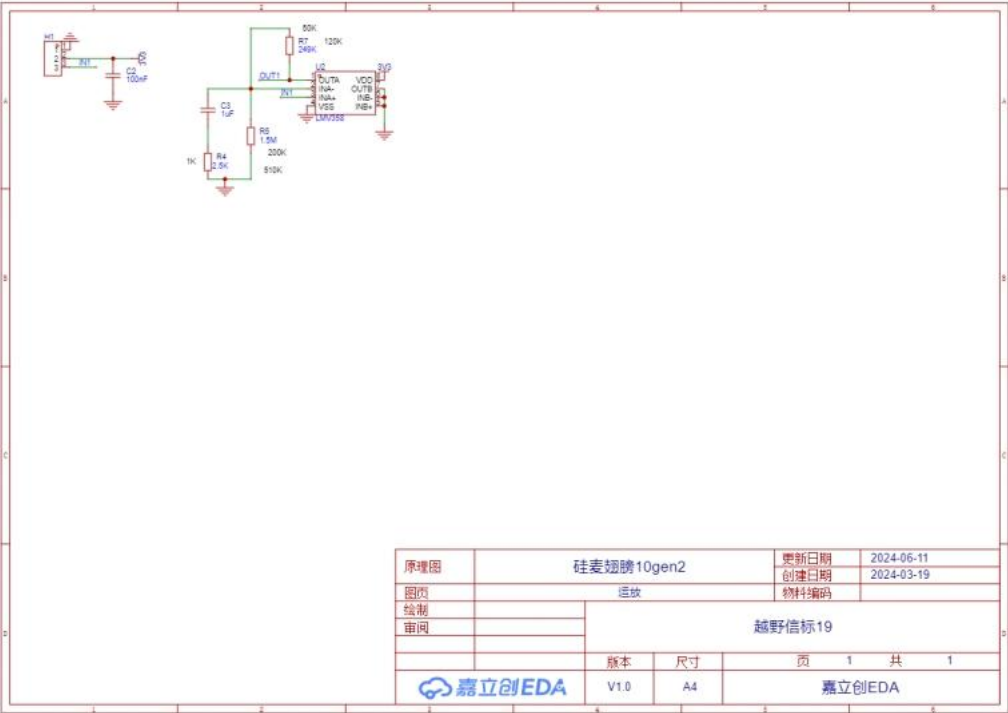


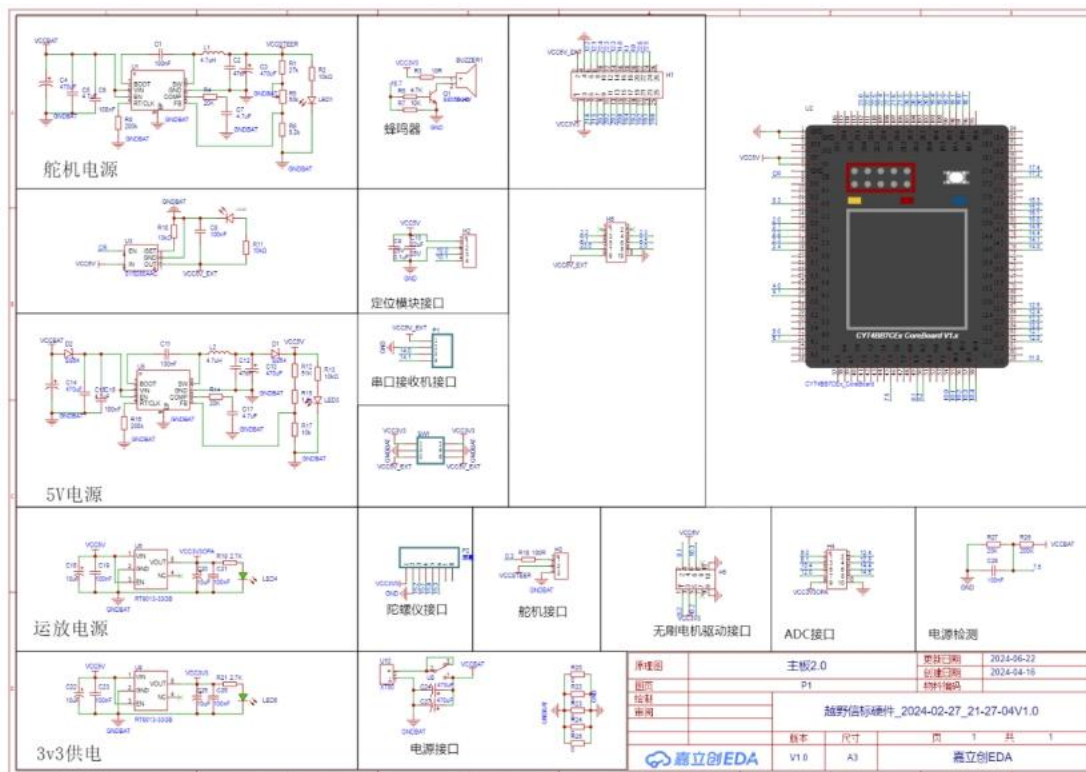
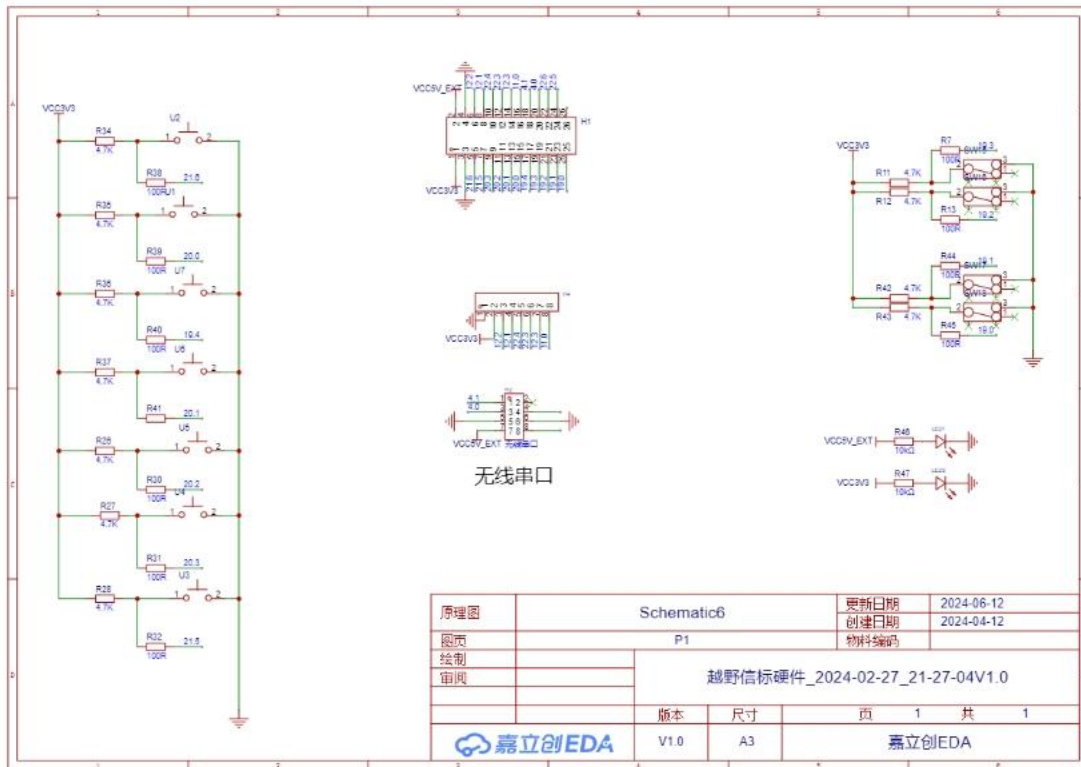
▲ 图 1.2 电路板照片示例
在图中标记处电路板本身的 LOGO 信息

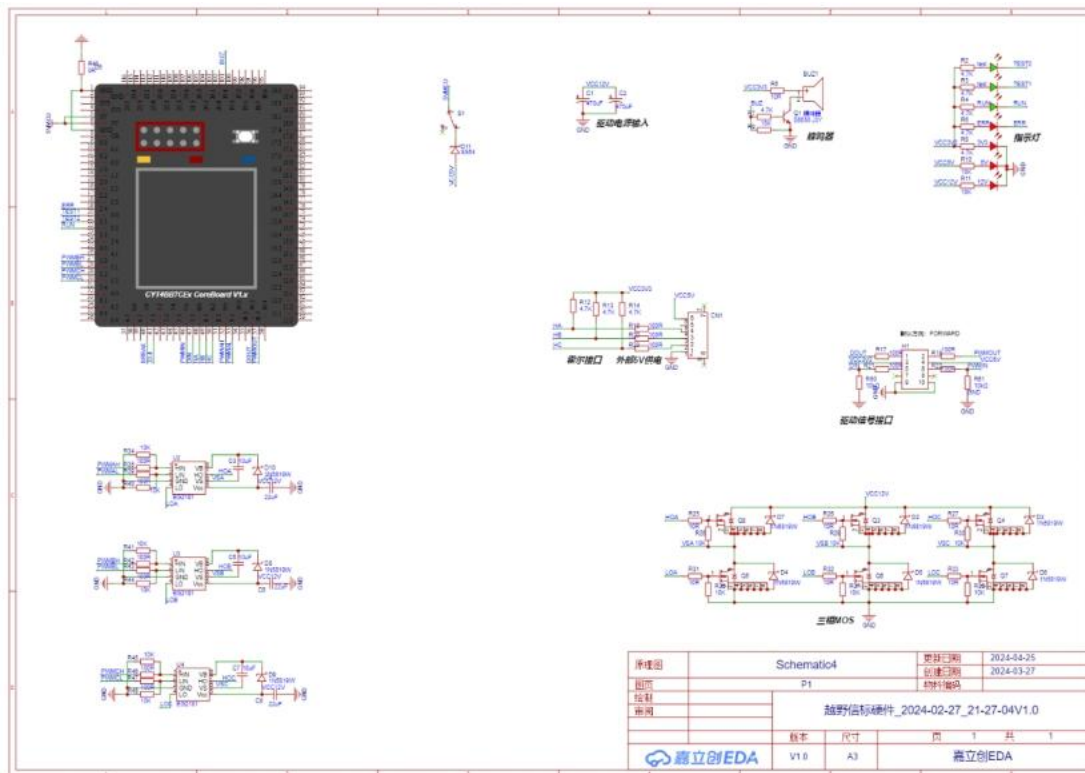
3、电路板原理图

请在生成电路板图形需要足够的分辨率，能够分辨出电路图元器件、引线、标注文字等信息。如果电路图过大，请将电路图拆成若干小的电路图分别生成图片。

下面的示例图片，由于电路图过大，生成的图片无法分辨电路图细节，不符合要求。







参考文献

- [1] 第十八届全国大学生智能汽车竞赛竞速比赛规则.
- [2] 王盼宝, 佟超等. 智能车制作. 北京, 清华大学出版社, 2001.
- [3] 谭浩强. C 程序设计(第二版). 北京, 清华大学出版社, 2001.
- [4] 贾超, 卜旭芳, 陈晓芳, 安文倩, 李昆. 飞思卡尔智能车竞赛赛道新规则解析[J]. 电子设计工程, 2015, 23(09):49-51. DOI:10.14022/j.cnki.dzs.jgc.2015.09.014.

附录：核心代码

主控板 main.c 代码

```
#include "zf_common_headfile.h"
#include "All_Init.h"

int main(void)
```

```

{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_info_init();              // 调试串口信息初始化
    system_delay_ms(500);
    // 此处编写用户代码 例如外设初始化代码等

    all_init();

    Bee();
    // timer_init(TC_TIME2_CH0,TIMER_US);
    // timer_start(TC_TIME2_CH0);
    pit_ms_init(PIT_CH0,5);

    while(true)
    {
        //SCB_CleanInvalidateDCache();
        if(motor_test_flag!=1)
            Screen();

        //printf("direction:%f\n",m7_1_data[0]);

        //SCREEN_SHOW_INT(0,0,encoder_get_count(TC_CH20_ENCODER),4);

        // if(K_MODE_2)
        // {
        //     oscilloscope_data.data[0] = Speed_want;
        //     oscilloscope_data.data[2] = distance_gps;
        //     oscilloscope_data.data[3] = speed_duty;
        //     oscilloscope_data.data[4] = m7_1_data[0];
        //     oscilloscope_data.data[1] = SpeedNow;
        //     oscilloscope_data.data[5] = m7_1_data[2];
        //     seekfree_assistant_oscilloscope_send(&oscilloscope_data);
        // }
    }
}

```

```
}
```

速度控制代码:

```
void speed_cal_gps(void)          //gps 模式下的速度决策
{
    float speed_high_gps_temp=0.0;
    float speed_want_temp=0;
    float k1=0;
    float KAngle=0;//角度系数
    if(temp[lora_receive_num].type%2==0)
        speed_high_gps_temp=speed_high_gps;
    else
        speed_high_gps_temp=speed_high_gps-2;

    if(distance_gps<=gps_distance_max&&distance_gps>gps_distance_min)
    {
        if(SpeedNow>speed_low_gps)    //当速度大于期望速度，改为输出固定电
压
            velocity_phase=CUT;
        else
            velocity_phase=UP;
        //        if(Bang_bang_flag)
        //        {
        //            speed_want_temp=speed_low_gps;
        //        }
        //        else
        if(Deceleration_curve_flag==0)    //速度曲线，一次函数
        {
            k1=(speed_high_gps_temp-speed_low_gps)/(gps_distance_max-
gps_distance_min);
            speed_want_temp=k1*(distance_gps-
gps_distance_min)+speed_low_gps;
        }
        else if(Deceleration_curve_flag==1)    //速度曲线，二次函数
        {
            k1=(distance_gps-gps_distance_min)/(gps_distance_max-
gps_distance_min);
```



```

        speed_want_temp=k1*k1*(speed_high_gps_temp-
speed_low_gps)+speed_low_gps;
    }
}
else if(distance_gps<=gps_distance_min)
{
    speed_want_temp=speed_low_gps;
    velocity_phase=SOUND;
}
else if(distance_gps>gps_distance_max)
{
    speed_want_temp=speed_high_gps_temp;
    velocity_phase=UP;
}

//      if(Buff[2]==2)
//          speed_want_temp=(float)(Buff[4]-0x64)/10;
if(fabs(err)>=straight_angle)    //舵机打角减速
    Speed_want=angle_low_speed;
else
{
    if(speed_want_temp)
    {
        KAngle=(speed_want_temp)/240;
        Speed_want=speed_want_temp-KAngle*abs(servo_motor_duty);
    }
}

}

uint8  stop_flag=0;
float speed_high_gps=8.0;
float speed_low_gps=3.0;
float speed_high_sound=3.0;
float speed_low_sound=2.0;

```

```

float gps_distance_max=6.0;    //开始减速距离值
float gps_distance_min=3.0;

float Speed_want=0.0;
float straight_angle=25;
float angle_low_speed=3.0;
void SpeedCal(void)
{
    if((run_flag&&run_mode==1&&finish_flag==0))    //当 GPS 模式
    {
        //      if(Bang_bang_flag)
        //      {
        //
        //      }
        //      else
            speed_cal_gps();
    }
    //  else if(run_flag&&finish_flag)
    //  {
    //      speed_cal_gps();
    //  }
    else if(run_flag==0&&(run_mode==0||run_mode==1||run_mode==2))
    {
        Speed_want=0;
        Speed_Target=0;
        Speed_Inter=0;
    }
}

float speed_increment=0.05;
float speed_increment_sub=0.2;
void motor_speed_curve(void)
{
    if(Speed_want>=0)

```

```

{
    if(Speed_want>=Speed_Target)
    {
        Speed_Target += speed_increment; //步进值 0.05M/S
        if(Speed_Target > Speed_want)
        {
            Speed_Target = Speed_want;
        }
    }
    //设置的速度比目标速度低，则不使用步进值
    else if(Speed_want < Speed_Target)
    {
        Speed_Target-=speed_increment_sub;
    }
    if(Speed_Target < Speed_want)
    {
        Speed_Target = Speed_want;
    }
}
else
{
    if(Speed_want>=Speed_Target)
    {
        Speed_Target =Speed_want;
    }
    //设置的速度比目标速度低，则不使用步进值
    else if(Speed_want < Speed_Target)
    {
        if(Bang_bang_flag)
            Speed_Target=Speed_want;
        else
        {
            Speed_Target -= speed_increment_sub; //步进值 0.075M/S
            if(Speed_Target < Speed_want)
            {
                Speed_Target = Speed_want;
            }
        }
    }
}
}

```

```

    }
    }
    }
}
}

```

人机交互代码:

```
#include "All_Init.h"
```

```
uint8 search_point_mode=1;
```

```
char page=0,hang=0,add=0,subtract=0,save=0,clc=0,k12=0;    //页，行，加，减
```

```
uint8 motor_test_flag=0;
```

```
uint8 random_flag=0;
```

```
uint8 angle_adjust_flag=0;
```

```
void key_scan_new(void)
```

```

{
    if(K_PAGE_ADD==0)  //换页
    {
        system_delay_ms(4);
        if(K_PAGE_ADD==0)
        {
            while(!K_PAGE_ADD);
            page=1;
        }
    }

    if(K_HANG_ADD==0)  //换行
    {
        system_delay_ms(4);
        if(K_HANG_ADD==0)
        {
            while(!K_HANG_ADD);
            hang=1;
        }
    }
}

```

```

}

if(K_HANG_SUB==0)  //换行
{
    system_delay_ms(4);
    if(K_HANG_SUB==0)
    {
        while(!K_HANG_SUB);
        hang=2;
    }
}

if(K_ADD==0)  //增加
{
    system_delay_ms(4);
    if(K_ADD==0)
    {
        while(!K_ADD);
        add=1;
    }
}

if(K_SUB==0)  //减少
{
    system_delay_ms(4);
    if(K_SUB==0)
    {
        while(!K_SUB);
        subtract=1;
    }
}

if(K_SAVE==0)  //减少
{
    system_delay_ms(4);
    if(K_SAVE==0)

```

```

        {
            while(!K_SAVE);
            save=1;
        }
    }

    if(K_CLC==0)  //换页
    {
        system_delay_ms(4);
        if(K_CLC==0)
        {
            while(!K_CLC);
            clc=1;
        }
    }

    if(K_PAGE_SUB==0)  //换页
    {
        system_delay_ms(4);
        if(K_PAGE_SUB==0)
        {
            while(!K_PAGE_SUB);
            page=2;
        }
    }

    if(K_12==0)  //换页
    {
        system_delay_ms(4);
        if(K_12==0)
        {
            while(!K_12);
            k12=1;
        }
    }
}

```


uint8 page_num=0,hang_num=0,hang_num_last=0,page_num_last=0; //页数，
行数，上次页数，上次行数

void display_01(void) //参数调整页面

```
{
    if(clc==1) //按下以后
    {
        clc=0;
        SCREEN_CLEAR();
        Bee();
    }

    if(page==1) //按下以后
    {
        page=0;
        page_num++;
        Bee();
        if(page_num>3) page_num=0;
    }

    if(page==2) //按下以后
    {
        page=0;
        Bee();
        if(page_num>0) page_num--;
        else
            page_num=3;
    }

    if(hang==1)
    {
        hang=0;
        hang_num++;
        Bee();
        if(hang_num>11) hang_num=2;
    }
}
```

```

if(hang==2)
{
    hang=0;
    hang_num--;
    Bee();
    if(hang_num<2) hang_num=9;
}

if(page_num==0)
{
    if(add==1)
    {
        add=0;
        Bee();
        switch(hang_num)
        {
            case(2) : search_point_mode++;                break;    //运动
模式 1-GPS&SOUND, 2-SOUND, 3-GPS
            case(3) : run_mode++;                          break;    //寻迹
模式 1-GPS, 2-SOUND
            case(4) : angle_adjust_flag++;                break;
            case(5) : random_flag++;                      break;
            case(6) : direction_flag++;                    break;
            case(7) : Deceleration_curve_flag++;          break;
            case(8) : daoche_flag++;                       break;
            case(9) : Open_loop_flag++;                    break;
            case(10) : Bang_bang_flag++;                    break;
        }
    }

    if(subtract==1)
    {
        subtract=0;
        Bee();
        switch(hang_num)

```

```

        {
            case(2) : search_point_mode--;      break;
            case(3) : run_mode--;                break;
            case(4) : angle_adjust_flag--;        break;
            case(5) : random_flag--;              break;
            case(6) : direction_flag--;          break;
            case(7) : Deceleration_curve_flag--; break;
            case(8) : daoche_flag--; break;
            case(9) : Open_loop_flag--;          break;
            case(10) : Bang_bang_flag--;         break;
        }
    }
}
if(page_num==1)
{
    if(add==1)
    {
        add=0;
        Bee();
        switch(hang_num)
        {
            case(2) : speed_high_gps+=0.5;      break;      //设定
速度上限
            case(3) : speed_low_gps+=0.5;        break;      //速度
下限
            case(4) : speed_high_sound+=0.5;     break;      //速度
下限
            case(5) : speed_low_sound+=0.5;      break;      //速度
下限
            case(8) : speed_increment+=0.005;    break;      //速度
步进值
            case(6) : gps_distance_max+=0.5;     break;      //舵
机 kp
            case(7) : gps_distance_min+=0.5;     break;
            case(9) : distance_temp+=0.5;        break;      //速度步
进值

```

```

        case(10) : servo_mid+=5;          break;
    }
}

if(subtract==1)
{
    subtract=0;
    Bee();
    switch(hang_num)
    {
        case(2) : speed_high_gps-=0.5;      break;      //设定速
度上限
        case(3) : speed_low_gps-=0.5;        break;      //速度
下限
        case(4) : speed_high_sound-=0.5;      break;      //速度
下限
        case(5) : speed_low_sound-=0.5;        break;      //速度
下限
        case(6) : speed_increment-=0.005;      break;      //速度
步进值
        case(7) : gps_distance_max-=0.5;        break;      //舵
机 kp
        case(8) : gps_distance_min-=0.5;        break;
        case(9) : distance_temp-=0.5;          break;      //速度步
进值
        case(10) : servo_mid-=2;          break;
    }
}
}

else if(page_num==2)
{
    if(add==1)
    {
        add=0;
        Bee();
        switch(hang_num)

```

```

        {
            case(2) : servo_kp+=0.05;          break;          //舵机 kp
            case(3) : servo_kd+=5;             break;          //舵机 kd
            case(4) : SpeedKp+=20;              break;          //电机 kd
            case(5) : SpeedKi+=1;              break;          //电机 ki
            case(6) : SpeedKd+=1;              break;          //电机
kd
            case(7) : servo_kp_sound+=0.1;      break;          //舵机
kp
            case(8) : servo_kd_sound+=5;        break;
            case(9) : straight_angle+=2;        break;
            case(10): angle_low_speed+=0.5;     break;
        }
    }

    if(subtract==1)
    {
        subtract=0;
        Bee();
        switch(hang_num)
        {
            case(2) : servo_kp-=0.05;          break;          //舵机 kp
            case(3) : servo_kd-=2;             break;          //舵机 kd
            case(4) : SpeedKp-=20;              break;          //电机 kd
            case(5) : SpeedKi-=1;              break;          //电机 ki
            case(6) : SpeedKd-=1;              break;          //电机 kd
            case(7) : servo_kp_sound-=0.1;      break;          //舵机
kp
            case(8) : servo_kd_sound-=2;        break;
            case(9) : straight_angle-=2;        break;
            case(10): angle_low_speed-=0.5;     break;
        }
    }
}

else if(page_num==3)

```

```

{
    if(add==1)
    {
        add=0;
        Bee();
        switch(hang_num)
        {
            case(2) : Bang_bang_value+=200;          break;          //舵
机 kp

            case(3) : cut_value+=200;          break;

            case(4) : Speed_Output_limit+=100;          break;          //
舵机 kp

            case(5) : Speed_Output_limit_-=100;          break;          //
舵机 kp

            case(6) : distance_point_max+=0.1;          break;
            case(7) : distance_point_min+=0.1;          break;
            case(8) : distance_update+=0.1;          break;
            case(9) : daoche_speed+=0.1;          break;
            case(10): delay_time+=20;          break;
        }
    }

    if(subtract==1)
    {
        subtract=0;
        Bee();
        switch(hang_num)
        {
            case(2) : Bang_bang_value-=200;          break;          //舵
机 kp

            case(3) : cut_value-=200;          break;

            case(4) : Speed_Output_limit-=100;          break;          //
舵机 kp

            case(5) : Speed_Output_limit_-=100;          break;          //
舵机 kp

            case(6) : distance_point_max-=0.1;          break;

```

```

        case(7) : distance_point_min-=0.1;      break;
        case(8) : distance_update-=0.1;        break;
        case(9) : daoche_speed-=0.1;           break;
        case(10): delay_time-=20;              break;
    }
}

if(hang_num!=hang_num_last)                    //消影
    SCREEN_SHOW_STRING(0*8,hang_num_last*16," ");
if(page_num!=page_num_last)                   //清屏
    SCREEN_CLEAR();
hang_num_last=hang_num;
page_num_last=page_num;
}

void Display_01(void)        //参数调整页面 ui
{
    SCREEN_SHOW_STRING(0,16*hang_num,"->");    //箭头上下移
    位
    if(page_num==0)
    {
        SCREEN_SHOW_STRING(2*8,2*16,"Point_Mode:");
        SCREEN_SHOW_INT(8*13,2*16,search_point_mode,1);
        SCREEN_SHOW_STRING(2*8,3*16,"Run_Mode:");
        SCREEN_SHOW_INT(8*13,3*16,run_mode,1);
        SCREEN_SHOW_STRING(2*8,4*16,"ANG_ADJ:");
        SCREEN_SHOW_INT(8*13,4*16,angle_adjust_flag,1);
        SCREEN_SHOW_STRING(2*8,5*16,"Random:");
        SCREEN_SHOW_INT(8*13,5*16,random_flag,1);
        SCREEN_SHOW_STRING(2*8,6*16,"Atan:");
        SCREEN_SHOW_INT(8*13,6*16,direction_flag,2);
        SCREEN_SHOW_STRING(2*8,7*16,"Deceler_flag:");
        SCREEN_SHOW_INT(8*13,7*16,Deceleration_curve_flag,2);
        //Deceleration_curve_flag
        SCREEN_SHOW_STRING(2*8,8*16,"daoche:");
    }
}

```



```

SCREEN_SHOW_INT(8*13,8*16,daoche_flag,2);
    SCREEN_SHOW_STRING(2*8,9*16,"Open_loop:");
SCREEN_SHOW_INT(8*13,9*16,Open_loop_flag,2);
    SCREEN_SHOW_STRING(2*8,10*16,"Bang_flag:");
SCREEN_SHOW_INT(8*13,10*16,Bang_bang_flag,2);
}

if(page_num==1)
{
    SCREEN_SHOW_STRING(2*8,2*16,"SP_H_GPS:");
SCREEN_SHOW_FLOAT(8*13,2*16,speed_high_gps,2,2);
    SCREEN_SHOW_STRING(2*8,3*16,"SP_L_GPS:");
SCREEN_SHOW_FLOAT(8*13,3*16,speed_low_gps,2,1);
    SCREEN_SHOW_STRING(2*8,4*16,"SP_H_SOU:");
SCREEN_SHOW_FLOAT(8*13,4*16,speed_high_sound,2,2);
    SCREEN_SHOW_STRING(2*8,5*16,"SP_L_SOU:");
SCREEN_SHOW_FLOAT(8*13,5*16,speed_low_sound,2,1);
    SCREEN_SHOW_STRING(2*8,6*16,"SPEED_I:");
SCREEN_SHOW_FLOAT(8*13,6*16,speed_increment,1,3);
    SCREEN_SHOW_STRING(2*8,7*16,"G_D_MAX:");
SCREEN_SHOW_FLOAT(8*13,7*16,gps_distance_max,2,2);
    SCREEN_SHOW_STRING(2*8,8*16,"G_D_MIN:");
SCREEN_SHOW_FLOAT(8*13,8*16,gps_distance_min,2,2);
    SCREEN_SHOW_STRING(2*8,9*16,"DIS_TEMP:");
SCREEN_SHOW_FLOAT(8*13,9*16,distance_temp,1,2);
    SCREEN_SHOW_STRING(2*8,10*16,"SERVO_MID:");
SCREEN_SHOW_INT(8*13,10*16,servo_mid,4);
    //SCREEN_SHOW_STRING(2*8,10*16,"Bang_value:");
SCREEN_SHOW_INT(8*13,10*16,Bang_bang_value,4);
}

if(page_num==2)
{
    SCREEN_SHOW_STRING(2*8,2*16,"SERVO_KP:");
SCREEN_SHOW_FLOAT(8*13,2*16,servo_kp,2,2);
    SCREEN_SHOW_STRING(2*8,3*16,"SERVO_KD:");
SCREEN_SHOW_FLOAT(8*13,3*16,servo_kd,2,2);

```

```

        SCREEN_SHOW_STRING(2*8,4*16,"SPEED_KP:");
SCREEN_SHOW_FLOAT(8*13,4*16,SpeedKp,3,1);
        SCREEN_SHOW_STRING(2*8,5*16,"SPEED_KI:");
SCREEN_SHOW_FLOAT(8*13,5*16,SpeedKi,2,1);
        SCREEN_SHOW_STRING(2*8,6*16,"SPEED_KD:");
SCREEN_SHOW_FLOAT(8*13,6*16,SpeedKd,2,1);
        SCREEN_SHOW_STRING(2*8,7*16,"KP_SOUND:");
SCREEN_SHOW_FLOAT(8*13,7*16,servo_kp_sound,2,2);
        SCREEN_SHOW_STRING(2*8,8*16,"KD_SOUND:");
SCREEN_SHOW_FLOAT(8*13,8*16,servo_kd_sound,2,2);
        SCREEN_SHOW_STRING(2*8,9*16,"S_A:");
SCREEN_SHOW_FLOAT(8*13,9*16,straight_angle,2,1);
        SCREEN_SHOW_STRING(2*8,10*16,"A_L_S:");
SCREEN_SHOW_FLOAT(8*13,10*16,angle_low_speed,2,1);
    }

    if(page_num==3)
    {
        SCREEN_SHOW_STRING(2*8,2*16,"Bang_value:");
SCREEN_SHOW_INT(8*13,2*16,Bang_bang_value,4);
        SCREEN_SHOW_STRING(2*8,3*16,"Bang_CUT:");
SCREEN_SHOW_INT(8*13,3*16,cut_value,4);
        SCREEN_SHOW_STRING(2*8,4*16,"Out_limit:");
SCREEN_SHOW_INT(8*13,4*16,Speed_Output_limit,4);
        SCREEN_SHOW_STRING(2*8,5*16,"Out_limit__");
SCREEN_SHOW_INT(8*13,5*16,Speed_Output_limit_,4);
        SCREEN_SHOW_STRING(2*8,6*16,"P_D_Max:");
SCREEN_SHOW_FLOAT(8*13,6*16,distance_point_max,2,1);
        SCREEN_SHOW_STRING(2*8,7*16,"P_D_Min:");
SCREEN_SHOW_FLOAT(8*13,7*16,distance_point_min,2,1);
        SCREEN_SHOW_STRING(2*8,8*16,"P_UPDATA_D:");
SCREEN_SHOW_FLOAT(8*13,8*16,distance_update,2,1);
        SCREEN_SHOW_STRING(2*8,9*16,"Back_speed:");
SCREEN_SHOW_FLOAT(8*13,9*16,daoche_speed,2,1);
        SCREEN_SHOW_STRING(2*8,10*16,"delay_time:");
SCREEN_SHOW_INT(8*13,10*16,delay_time,4);
    }

```

```
    }  
}
```

```
uint8 point_type=0,point_num=0;
```

```
void display_11(void)                //存点页面
```

```
{
```

```
    if(clc==1)
```

```
    {
```

```
        SCREEN_CLEAR();
```

```
        clc=0;
```

```
        Bee();
```

```
    }
```

```
    if(hang==1)
```

```
    {
```

```
        hang=0;
```

```
        hang_num++;
```

```
        Bee();
```

```
        if(hang_num>4) hang_num=2;
```

```
    }
```

```
    if(hang==2)
```

```
    {
```

```
        hang=0;
```

```
        hang_num--;
```

```
        Bee();
```

```
        if(hang_num<2) hang_num=4;
```

```
    }
```

```
    if(add==1)
```

```
    {
```

```
        add=0;
```

```
        Bee();
```

```
        switch(hang_num)
```

```

        {
            case(2) : the_point++;          break;          //当前数组
下标
            case(3) : save_mode++;          break;          //当前点
位对应的灯编号，默认与下标相同
            case(4) : temp[the_point].type++; break;          //当前点位
对应的灯类型 偶数为自己跑，奇数别的车跑
        }
    }

```

```

    if(subtract==1)
    {
        subtract=0;
        Bee();
        switch(hang_num)
        {
            case(2) : the_point--;          break;
            case(3) : save_mode--;          break;
            case(4) : temp[the_point].type--; break;
        }
    }

```

```

    if(hang_num!=hang_num_last)
        SCREEN_SHOW_STRING(0*8,hang_num_last*16," ");
    if(page_num!=page_num_last)
        SCREEN_CLEAR();
    hang_num_last=hang_num;
    page_num_last=page_num;
}

```

```

uint8 save_mode=0;
void save_point(void)          //存点
{
    if(save==1)                //中按键存实际点
    {

```

```

if(save_mode==0)
{
    temp[the_point].lat=gnss.latitude;
    temp[the_point].lon=gnss.longitude;
    temp[the_point].Num=the_point;
    the_point++;
}
else if(save_mode==1)
{
    save_point_average[point_num].lat=gnss.latitude;
    save_point_average[point_num].lon=gnss.longitude;
    point_num++;
    if(point_num>=4)
    {
        point_num=0;

temp[the_point].lat=save_point_average[0].lat+save_point_average[1].lat+save_p
oint_average[2].lat+save_point_average[3].lat;

temp[the_point].lon=save_point_average[0].lon+save_point_average[1].lon+save
_point_average[2].lon+save_point_average[3].lon;
        temp[the_point].lat/=4.0;
        temp[the_point].lon/=4.0;

        the_point++;
    }
}
save=0;
Bee();
}
if(page==1)        //右上存虚拟点
{
    if(save_mode==0)
    {
        temp[the_point].lat_before=gnss.latitude;
        temp[the_point].lon_before=gnss.longitude;

```

```

        temp[the_point].Num=the_point;
        the_point++;
    }
    else if(save_mode==1)
    {
        save_point_average[point_num].lat=gnss.latitude;
        save_point_average[point_num].lon=gnss.longitude;
        point_num++;
        if(point_num>=3)
        {
            point_num=0;

temp[the_point].lat_before=save_point_average[0].lat+save_point_average[1].lat
+save_point_average[2].lat+save_point_average[3].lat;

temp[the_point].lon_before=save_point_average[0].lon+save_point_average[1].lon
n+save_point_average[2].lon+save_point_average[3].lon;
            temp[the_point].lat_before/=3.0;
            temp[the_point].lon_before/=3.0;

            the_point++;
        }
    }
    page=0;
    Bee();
}
}

```

```

void Display_11(void)          //存点页面 ui
{
    SCREEN_SHOW_STRING(0,16*hang_num,"->");
    SCREEN_SHOW_STRING(2*8,2*16,"T_P:");
    SCREEN_SHOW_INT(8*13,2*16,the_point,1);
    SCREEN_SHOW_STRING(2*8,3*16,"SAVE_MODE:");
    SCREEN_SHOW_INT(8*13,3*16,save_mode,1);
    SCREEN_SHOW_STRING(2*8,4*16,"T_T:");
}

```

```

if(temp[the_point].type%2)          SCREEN_SHOW_STRING(8*8,4*16,"OTHER");
else  SCREEN_SHOW_STRING(8*8,4*16,"SELF");
      SCREEN_SHOW_STRING(2*8,5*16,"LO:");
SCREEN_SHOW_FLOAT(8*13,5*16,temp[the_point].lon,2,3);
      SCREEN_SHOW_STRING(2*8,6*16,"LA:");
SCREEN_SHOW_FLOAT(8*13,6*16,temp[the_point].lat,2,3);
      SCREEN_SHOW_STRING(2*8,7*16,"NUM:");
SCREEN_SHOW_INT(8*13,7*16,gnss.satellite_used,2);

```

```

      float distance=get_two_points_distance(gnss.latitude      ,gnss.longitude      ,
temp[the_point].lat      ,temp[the_point].lon);//距目的地距离;
      SCREEN_SHOW_STRING(0,16*8,"DISTANCE");
SCREEN_SHOW_FLOAT(8*13,8*16,distance,2,3);

```

```

      SCREEN_SHOW_STRING(0,16*9,"Direction");
SCREEN_SHOW_FLOAT(8*13,9*16,gnss.direction,3,2);

```

```

      float
azimution=get_two_points_azimuth(gnss.latitude      ,gnss.longitude      ,
temp[the_point].lat      ,temp[the_point].lon);//距目的地距离;
      SCREEN_SHOW_STRING(0,16*10,"Azimution");
SCREEN_SHOW_FLOAT(8*13,10*16,azimution,3,3);

```

```

      float
distance_before=get_two_points_distance(gnss.latitude      ,gnss.longitude      ,
temp[the_point].lat_before      ,temp[the_point].lon_before);//距目的地距离;
      SCREEN_SHOW_STRING(0,16*11,"DISTANCE_Bef");
SCREEN_SHOW_FLOAT(8*14,11*16,distance_before,2,3);
      SCREEN_SHOW_STRING(0,16*12,"DIR_ICM");
SCREEN_SHOW_FLOAT(8*14,12*16,direction_icm,3,2);
}

```

```

void display_00(void)  //发车页面
{
    if(clc==1)        //按下以后
    {

```

```

        clc=0;
        SCREEN_CLEAR();
        Bee();
        Stop_car();
        //stop_flag=0;
        motor_test_distance_temp=0;
        finish_flag=0;
        start_flag=0;
        stop_flag=0;
        point_progress=0;
    }

    if(save==1)
    {
        save=0;
        Bee();
        system_delay_ms(2000);
        motor_test_flag=1;
        run_flag=1;
    }

    if(hang==1)
    {
        hang=0;
        hang_num++;
        Bee();
        if(hang_num>5) hang_num=2;
    }

    if(hang==2)
    {
        hang=0;
        hang_num--;
        Bee();
        if(hang_num<2) hang_num=5;
    }

```



```

        if(add==1)
        {
            add=0;
            Bee();
            switch(hang_num)
            {
                case(2) : motor_test_distance+=10;                break;
//当前数组下标
                case(3) : Speed_want+=1;        break;        //当前点位对应的
灯编号，默认与下标相同
                case(4) : start_flag+=1;        break;        //当前点位对应的灯类
型 偶数为自己跑，奇数别的车跑
                case(5) : test_duty+=500;  break;
            }
        }

```

```

        if(subtract==1)
        {
            subtract=0;
            Bee();
            switch(hang_num)
            {
                case(2) : motor_test_distance-=10;                break;
                case(3) : Speed_want-=1;        break;
                case(4) : start_flag-=1;        break;
                case(5) : test_duty-=500;  break;
            }
        }

```

```

if(hang_num!=hang_num_last)
    SCREEN_SHOW_STRING(0*8,hang_num_last*16," ");
if(page_num!=page_num_last)
    SCREEN_CLEAR();
hang_num_last=hang_num;

```

```

    page_num_last=page_num;

    if(page==1)
    {
        page=0;
        Bee();
        motor_test_distance+=40;
    }
}

void Display_1(void)    //发车页面 ui
{
    //    if(motor_test_flag==1)
    //        SCREEN_SHOW_STRING(0,3*16,"START_TEST");
    //    else
    //        SCREEN_SHOW_STRING(0,3*16,"NO_TEST");
    SCREEN_SHOW_STRING(0,16*hang_num,"->");
    SCREEN_SHOW_STRING(16,2*16,"DIS");
    SCREEN_SHOW_FLOAT(8*13,2*16,motor_test_distance_temp,3,1);
    SCREEN_SHOW_STRING(16,3*16,"SP");
    SCREEN_SHOW_FLOAT(8*13,3*16,Speed_want,2,2);
    SCREEN_SHOW_STRING(16,4*16,"St_P");
    SCREEN_SHOW_FLOAT(8*13,4*16,start_flag,3,1);
    SCREEN_SHOW_STRING(16,6*16,"Dir_zf");
    SCREEN_SHOW_FLOAT(8*13,6*16,m7_1_data[0],3,1);
    SCREEN_SHOW_STRING(16,7*16,"Dir_my");
    SCREEN_SHOW_FLOAT(8*13,7*16,m7_1_data[2],3,1);
    SCREEN_SHOW_STRING(16,5*16,"Test_Duty");
    SCREEN_SHOW_FLOAT(8*13,5*16,test_duty,4,1);
    SCREEN_SHOW_STRING(16,8*16,"Voltage");
    SCREEN_SHOW_FLOAT(8*13,8*16,voltage_value,4,2);

}

void Display_00(void)
{

```

```

switch(search_point_mode)
{
    case(0) : SCREEN_SHOW_STRING(0,0,"NO_SET");           break;
    case(1) : SCREEN_SHOW_STRING(0,0,"GPS&Sound_mode");   break;
    case(2) : SCREEN_SHOW_STRING(0,0,"Sound_mode");       break;
    case(3) : SCREEN_SHOW_STRING(0,0,"GPS_Mode");         break;
    case(4) : SCREEN_SHOW_STRING(0,0,"Control_mode");     break;
}

switch(run_mode)
{
    case(1) : SCREEN_SHOW_STRING(0,16*1,"GPS");           break;
    case(2) : SCREEN_SHOW_STRING(0,16*1,"SOUND");         break;
}

switch(direction_flag)
{
    case(1) : SCREEN_SHOW_STRING(8*10,16*1,"zf");         break;
    case(2) : SCREEN_SHOW_STRING(8*10,16*1,"my");         break;
}
}

uint8 order[30]={0};           //用于存储灯响的顺序
uint8 order_num=0;
void display_10(void)          //存信标灯数据页面
{
    if(page==2)
    {
        order[order_num]=1;
        order_num++;
        Bee();
        page=0;
        SCREEN_CLEAR();
    }

    if(hang==2)

```

```
{  
    order[order_num]=2;  
    order_num++;  
    Bee();  
    hang=0;  
    SCREEN_CLEAR();  
}
```

```
if(page==1)  
{  
    order[order_num]=3;  
    order_num++;  
    Bee();  
    page=0;  
    SCREEN_CLEAR();  
}
```

```
if(subtract==1)  
{  
    order[order_num]=4;  
    order_num++;  
    Bee();  
    subtract=0;  
    SCREEN_CLEAR();  
}
```

```
if(save)  
{  
    order[order_num]=5;  
    order_num++;  
    Bee();  
    save=0;  
    SCREEN_CLEAR();  
}
```

```
if(add)
```

```

{
    order[order_num]=6;
    order_num++;
    Bee();
    add=0;
    SCREEN_CLEAR();
}

if(k12)
{
    order[order_num]=7;
    order_num++;
    Bee();
    k12=0;
    SCREEN_CLEAR();
}

if(hang==1)
{
    order[order_num]=8;
    order_num++;
    Bee();
    hang=0;
    SCREEN_CLEAR();
}

if(clc)
{
    for(uint8 i=0;i<30;i++)
        order[i]=0;
    order_num=0;
    clc=0;
    Bee();
    SCREEN_CLEAR();
}
}

```

```

void Display_10(void)          //存信标灯数据页面 ui
{
    for(uint8 j=0;j<5;j++)
    {
        for(uint8 i=0;i<6;i++)
        {
            SCREEN_SHOW_INT(i*30,j*32,order[i+j*6],1);
        }
    }
    SCREEN_SHOW_STRING(order_num%6*30,(order_num/6)*32+16,"!");
    SCREEN_SHOW_STRING(8*5,16*14,"order:");
    SCREEN_SHOW_INT(8*14,16*14,order_num+1,2);
}

void Screen(void)             //调用
{
    key_scan_new();

    if(K_MODE_1)
    {
        Display_00();
        display_01();
        Display_01();
    }
    if(K_MODE_3)
    {
        Display_00();
        save_point();
        display_11();
        Display_11();
    }

    if(K_MODE_0)

```

```
{
    Display_00();
    display_00();
    Display_1();
}

if(K_MODE_2)
{
    display_10();
    Display_10();
}

if(stop_flag)
    SCREEN_SHOW_STRING(0,16*13,"STOP!!");
}
```